

Universidade Federal da Bahia

Matheus Silva Freitas

# Blockchain como Sistema Distribuído

*Consenso, criptografia, Bitcoin, contratos inteligentes e Ethereum*

Salvador, Bahia

2026

# Resumo

Esta monografia explica blockchain a partir de sua base técnica: um sistema distribuído que mantém um estado compartilhado entre participantes independentes. O trabalho apresenta a origem da tecnologia, sua relação com redes ponto-a-ponto, criptografia, funções de hash, assinaturas digitais e mecanismos de consenso. Em seguida, analisa como Bitcoin organiza transações, blocos, carteiras, mineração e validação sem depender de uma autoridade central. Por fim, mostra a evolução para contratos inteligentes e Ethereum, destacando a Ethereum Virtual Machine, o uso de gas, as aplicações descentralizadas, DAOs e oráculos. O objetivo é demonstrar por que consenso, replicação e verificação criptográfica são elementos centrais da tecnologia blockchain, evitando reduzi-la apenas ao conceito de criptomoeda.

**Palavras-chave:** blockchain; sistemas distribuídos; consenso; criptografia; Bitcoin; Ethereum.

# Declaração de uso de inteligência artificial

Este trabalho contou com o auxílio de ferramentas de inteligência artificial generativa para sumarização de materiais de estudo, organização preliminar de conteúdo e ajustes textuais de clareza, coesão e revisão linguística. A seleção das fontes, a interpretação do conteúdo, a estrutura argumentativa e a responsabilidade acadêmica pelo texto final permanecem sob responsabilidade do autor, em conformidade com orientações contemporâneas de transparência no uso de ferramentas de apoio em produções científicas.

# Sumário

|                                                                   |           |
|-------------------------------------------------------------------|-----------|
| <b>Resumo</b>                                                     | <b>i</b>  |
| <b>Declaração de uso de inteligência artificial</b>               | <b>ii</b> |
| <b>1 Introdução às Blockchains e Sistemas Distribuídos</b>        | <b>1</b>  |
| 1.1 Contexto e Motivação . . . . .                                | 1         |
| 1.2 Origens e Evolução Histórica . . . . .                        | 1         |
| 1.2.1 Os Precursores: Criptografia e Moedas Eletrônicas . . . . . | 1         |
| 1.2.2 O Bitcoin e a Invenção do Blockchain . . . . .              | 2         |
| 1.2.3 Da Criptomoeda à Tecnologia de Propósito Geral . . . . .    | 2         |
| 1.3 Sistemas Distribuídos: Fundamentos . . . . .                  | 3         |
| 1.3.1 Definição e Características . . . . .                       | 3         |
| 1.3.2 O Problema dos Generais Bizantinos . . . . .                | 3         |
| 1.3.3 O Teorema CAP . . . . .                                     | 4         |
| 1.3.4 O Teorema PACELC . . . . .                                  | 4         |
| 1.4 Arquitetura Geral de uma Blockchain . . . . .                 | 5         |
| 1.4.1 Definições Fundamentais . . . . .                           | 5         |
| 1.4.2 Visão em Camadas . . . . .                                  | 5         |
| 1.4.3 Elementos Genéricos . . . . .                               | 6         |
| 1.4.4 O Processo de Consenso e Adição de Blocos . . . . .         | 7         |
| <b>2 Descentralização e Arquitetura de Rede</b>                   | <b>8</b>  |
| 2.1 O Conceito de Descentralização . . . . .                      | 8         |
| 2.2 Benefícios da Descentralização . . . . .                      | 9         |
| 2.2.1 Resiliência e Eliminação do Ponto Único de Falha . . . . .  | 9         |
| 2.2.2 Resistência à Censura . . . . .                             | 9         |
| 2.2.3 Transparência e Confiança Algorítmica . . . . .             | 9         |
| 2.3 Rede Peer-to-Peer: Topologia e Comunicação . . . . .          | 9         |
| 2.3.1 Topologia da Rede . . . . .                                 | 9         |
| 2.3.2 Propagação de Dados e o Protocolo Gossip . . . . .          | 10        |
| 2.4 Descentralização em Prática . . . . .                         | 10        |

|          |                                                           |           |
|----------|-----------------------------------------------------------|-----------|
| 2.4.1    | Contratos Inteligentes . . . . .                          | 10        |
| 2.4.2    | Organizações Autônomas Descentralizadas . . . . .         | 11        |
| 2.4.3    | Agentes Autônomos . . . . .                               | 11        |
| 2.5      | O Dilema da Descentralização . . . . .                    | 11        |
| 2.6      | Tipos de Blockchain por Acesso . . . . .                  | 12        |
| 2.6.1    | Blockchains Públicas . . . . .                            | 12        |
| 2.6.2    | Blockchains Privadas e de Consórcio . . . . .             | 12        |
| 2.6.3    | Blockchains Semi-Privadas e Híbridas . . . . .            | 12        |
| 2.6.4    | Ledgers Permissionados . . . . .                          | 13        |
| 2.7      | Tipos por Token . . . . .                                 | 13        |
| 2.8      | Arquitetura em Camadas . . . . .                          | 13        |
| 2.8.1    | Blockchains de Camada 1 . . . . .                         | 13        |
| 2.8.2    | Blockchains de Camada 2 . . . . .                         | 14        |
| <b>3</b> | <b>Fundamentos Criptográficos</b>                         | <b>15</b> |
| 3.1      | Introdução à Criptografia . . . . .                       | 15        |
| 3.1.1    | Serviços Providos pela Criptografia . . . . .             | 15        |
| 3.2      | Primitivas Criptográficas . . . . .                       | 16        |
| 3.2.1    | Primitivas Sem Chave . . . . .                            | 16        |
| 3.2.2    | Primitivas de Chave Simétrica . . . . .                   | 16        |
| 3.2.3    | Primitivas de Chave Assimétrica . . . . .                 | 17        |
| 3.3      | Funções de Hash . . . . .                                 | 17        |
| 3.3.1    | Propriedades das Funções de Hash . . . . .                | 17        |
| 3.3.2    | SHA-256 e Aplicações em Blockchain . . . . .              | 17        |
| 3.3.3    | Árvore de Merkle . . . . .                                | 18        |
| 3.4      | Criptografia Simétrica . . . . .                          | 18        |
| 3.5      | Criptografia Assimétrica . . . . .                        | 19        |
| 3.5.1    | Fundamentos Matemáticos . . . . .                         | 19        |
| 3.6      | Chaves Pública e Privada: Geração e Segurança . . . . .   | 19        |
| 3.7      | Assinaturas Digitais . . . . .                            | 20        |
| 3.8      | Endereços em Blockchain . . . . .                         | 20        |
| <b>4</b> | <b>Algoritmos de Consenso</b>                             | <b>21</b> |
| 4.1      | O Problema do Consenso em Sistemas Distribuídos . . . . . | 21        |
| 4.1.1    | Requisitos Fundamentais . . . . .                         | 22        |
| 4.1.2    | Tolerância a Falhas . . . . .                             | 22        |
| 4.2      | Falhas Bizantinas . . . . .                               | 23        |
| 4.3      | Proof of Work . . . . .                                   | 23        |
| 4.3.1    | Mecanismo e Busca do Nonce . . . . .                      | 23        |
| 4.3.2    | Target Hash e Dificuldade Ajustável . . . . .             | 24        |

|          |                                                                 |           |
|----------|-----------------------------------------------------------------|-----------|
| 4.3.3    | Segurança por Custo Computacional . . . . .                     | 24        |
| 4.4      | Proof of Stake . . . . .                                        | 24        |
| 4.4.1    | Validadores e Stake como Garantia . . . . .                     | 24        |
| 4.4.2    | Seleção de Validadores . . . . .                                | 25        |
| 4.4.3    | Slashing e Recompensas . . . . .                                | 25        |
| 4.4.4    | O Problema do Nothing at Stake . . . . .                        | 25        |
| 4.5      | Comparação entre PoW e PoS . . . . .                            | 26        |
| 4.6      | Trade-offs de Consenso . . . . .                                | 26        |
| 4.6.1    | O Teorema CAP Revisitado . . . . .                              | 26        |
| 4.6.2    | O Triângulo Impossível . . . . .                                | 27        |
| 4.6.3    | Finalidade: Probabilística versus Determinística . . . . .      | 27        |
| <b>5</b> | <b>Bitcoin — A Pioneira</b>                                     | <b>28</b> |
| 5.1      | Contexto Histórico e o Whitepaper de Satoshi Nakamoto . . . . . | 28        |
| 5.2      | Criptografia de Chaves e Posse de Bitcoin . . . . .             | 29        |
| 5.2.1    | Chave Privada . . . . .                                         | 29        |
| 5.2.2    | Chave Pública . . . . .                                         | 30        |
| 5.2.3    | Endereço Bitcoin . . . . .                                      | 30        |
| 5.2.4    | Custódia e Carteiras . . . . .                                  | 31        |
| 5.3      | Estrutura do Bloco Bitcoin . . . . .                            | 31        |
| 5.3.1    | Cabeçalho do Bloco . . . . .                                    | 31        |
| 5.3.2    | Corpo do Bloco e a Transação Coinbase . . . . .                 | 31        |
| 5.3.3    | Encadeamento e Imutabilidade . . . . .                          | 32        |
| 5.4      | Transações e o Modelo UTXO . . . . .                            | 32        |
| 5.4.1    | O Modelo UTXO . . . . .                                         | 32        |
| 5.4.2    | Estrutura de uma Transação . . . . .                            | 32        |
| 5.4.3    | Scripts e P2PKH . . . . .                                       | 33        |
| 5.4.4    | Ciclo de Vida de uma Transação e a Mempool . . . . .            | 33        |
| 5.5      | Mineração e Prova de Trabalho . . . . .                         | 34        |
| 5.5.1    | O Problema do Nonce e o Alvo de Dificuldade . . . . .           | 34        |
| 5.5.2    | Ajuste de Dificuldade . . . . .                                 | 34        |
| 5.5.3    | Recompensa e Halving . . . . .                                  | 34        |
| 5.5.4    | Evolução do Hardware de Mineração . . . . .                     | 35        |
| 5.5.5    | Pools de Mineração . . . . .                                    | 35        |
| 5.6      | A Cadeia: Encadeamento, Imutabilidade e Reorganização . . . . . | 35        |
| 5.6.1    | Encadeamento por Hash . . . . .                                 | 35        |
| 5.6.2    | Forks e Reorganização . . . . .                                 | 36        |
| 5.7      | A Rede Peer-to-Peer . . . . .                                   | 36        |
| 5.7.1    | Descoberta e Propagação . . . . .                               | 36        |

---

|          |                                                                       |           |
|----------|-----------------------------------------------------------------------|-----------|
| 5.7.2    | Tipos de Nós . . . . .                                                | 36        |
| 5.7.3    | Bloom Filters e Privacidade em SPV . . . . .                          | 37        |
| 5.8      | Carteiras Bitcoin . . . . .                                           | 37        |
| 5.8.1    | Tipos de Carteiras . . . . .                                          | 37        |
| 5.8.2    | HD Wallets, BIP32/39/44 e Seed Phrases . . . . .                      | 38        |
| 5.9      | Limitações e Críticas ao Bitcoin . . . . .                            | 39        |
| 5.9.1    | Throughput e Escalabilidade . . . . .                                 | 39        |
| 5.9.2    | Consumo Energético . . . . .                                          | 39        |
| 5.9.3    | Programabilidade Limitada . . . . .                                   | 39        |
| 5.9.4    | Privacidade . . . . .                                                 | 40        |
| 5.9.5    | Vulnerabilidades e Bugs Históricos . . . . .                          | 40        |
| <b>6</b> | <b>Contratos Inteligentes, Aplicações Descentralizadas e Oráculos</b> | <b>41</b> |
| 6.1      | Da Blockchain como Registro à Blockchain Programável . . . . .        | 41        |
| 6.2      | Propriedades dos Contratos Inteligentes . . . . .                     | 42        |
| 6.3      | Smart Contracts no Bitcoin e no Ethereum . . . . .                    | 42        |
| 6.4      | Aplicações Descentralizadas . . . . .                                 | 43        |
| 6.5      | DAOs e o Problema da Governança . . . . .                             | 43        |
| 6.6      | Oráculos e a Ponte com o Mundo Real . . . . .                         | 44        |
| <b>7</b> | <b>Arquitetura do Ethereum</b>                                        | <b>45</b> |
| 7.1      | Ethereum como Máquina de Estados Programável . . . . .                | 45        |
| 7.2      | Contas, Chaves e Endereços . . . . .                                  | 45        |
| 7.3      | Transações, Mensagens e Gas . . . . .                                 | 46        |
| 7.4      | Ethereum Virtual Machine . . . . .                                    | 46        |
| 7.5      | Estruturas de Dados e Estado . . . . .                                | 47        |
| 7.6      | Consenso e a Transição para Proof of Stake . . . . .                  | 47        |
| 7.7      | Tokens, Padrões e Ecossistema . . . . .                               | 47        |
| <b>8</b> | <b>Conclusão</b>                                                      | <b>49</b> |

# Capítulo 1

## Introdução às Blockchains e Sistemas Distribuídos

### 1.1 Contexto e Motivação

A tecnologia blockchain emergiu como uma das inovações mais discutidas e potencialmente transformadoras do início do século XXI. Embora frequentemente associada de maneira simplista às criptomoedas, seu alcance é consideravelmente mais amplo: trata-se de uma infraestrutura capaz de reconfigurar os paradigmas de confiança, transparência e coordenação em praticamente qualquer setor que dependa de registros compartilhados e de acordos entre partes que não necessariamente se conhecem ou confiam umas nas outras. Na síntese de Bashir (2023), blockchain é uma tecnologia que tem potencial para alterar positivamente os paradigmas existentes em setores que incluem finanças, governo, saúde, direito e tecnologia da informação, tornando-os mais eficientes, confiáveis e transparentes.

Compreender blockchain exige, necessariamente, partir de suas raízes intelectuais e técnicas. A tecnologia não surgiu em um vácuo: ela é o resultado de décadas de pesquisa em criptografia, sistemas distribuídos, protocolos de rede ponto-a-ponto e moedas digitais. O presente capítulo percorre esse trajeto histórico e conceitual, apresentando os fundamentos sobre os quais qualquer análise mais aprofundada deve se assentar.

### 1.2 Origens e Evolução Histórica

#### 1.2.1 Os Precursores: Criptografia e Moedas Eletrônicas

A história de blockchain não começa em 2008. Ela remonta a trabalhos de criptografia da década de 1970, quando Diffie e Hellman formularam métodos para a troca segura de chaves criptográficas, e à invenção subsequente da criptografia de chave pública em 1978. Em 1979, Ralph C. Merkle propôs as árvores de hash, estrutura que viria a



ser fundamental para a organização eficiente de transações dentro de blocos. Em 1982, Lamport et al. formalizaram o Problema dos Generais Bizantinos, um experimento mental que capturou de maneira precisa o desafio central de coordenar ações entre agentes independentes em um ambiente potencialmente hostil Bashir (2023).

A ideia de dinheiro eletrônico tampouco é nova. David Chaum trabalhou, na década de 1980, com assinaturas cegas e compartilhamento de segredos para criar sistemas de dinheiro digital que preservassem o anonimato dos usuários. Em 1992, Cynthia Dwork e Moni Naor publicaram o primeiro uso reconhecido de Prova de Trabalho (*Proof of Work*, PoW) como mecanismo de combate a spam. Em 1998, Nick Szabo propôs o Bit Gold, um mecanismo de moeda digital descentralizada que utilizava encadeamento de hashes e quóruns bizantinos. Em 2002, Adam Back formalizou o Hashcash, e em 2004 Hal Finney desenvolveu o sistema de PoW reutilizável. Todas essas contribuições convergiram para o surgimento do Bitcoin Bashir (2023).

### 1.2.2 O Bitcoin e a Invenção do Blockchain

Em 2008, um artigo intitulado *Bitcoin: A Peer-to-Peer Electronic Cash System*, publicado sob o pseudônimo de Satoshi Nakamoto, introduziu uma solução inédita para o problema do consenso distribuído em ambientes sem permissão, i.e., ambientes em que os participantes são anônimos e não existem mecanismos legais ou institucionais de responsabilização. A implementação prática ocorreu em 2009, com a mineração do bloco gênese da rede Bitcoin. O artigo introduziu a expressão *cadeia de blocos* (*chain of blocks*), que evoluiu para o termo *blockchain* Bashir (2023).

O problema central que o Bitcoin solucionou foi o do gasto duplo em um sistema sem autoridade central. Em sistemas de dinheiro digital pré-Bitcoin, a cópia de dados digitais tornava trivialmente possível gastar a mesma unidade monetária mais de uma vez. As soluções existentes dependiam de um servidor central confiável que mantivesse o registro de quais moedas já haviam sido gastas, o que introduzia um ponto único de falha e de controle. Bitcoin eliminou essa dependência ao substituir o árbitro central por um mecanismo de consenso distribuído ancorado em custo computacional verificável.

### 1.2.3 Da Criptomoeda à Tecnologia de Propósito Geral

Por volta de 2013 e 2014, pesquisadores e desenvolvedores começaram a perceber que a arquitetura subjacente ao Bitcoin poderia ser generalizada para aplicações que nada tinham a ver com transferência de valor monetário. Em 2015, o lançamento do Ethereum consolidou essa visão ao introduzir uma blockchain programável, capaz de executar contratos inteligentes (*smart contracts*): programas arbitrários que rodam de forma determinística sobre a rede distribuída. Esse desenvolvimento desbloqueou uma vastidão de possibilidades que incluem, entre outras, finanças descentralizadas (DeFi), tokens não

fungíveis (NFTs), organizações autônomas descentralizadas (DAOs) e aplicações descentralizadas (DApps) Bashir (2023).

A trajetória de maturidade tecnológica de blockchain assemelha-se, em ritmo e intensidade, à explosão da internet no final dos anos 1990. Após 2018, arquiteturas monolíticas e multicadeia evoluíram rapidamente, com protocolos como Solana, Avalanche e Polkadot introduzindo novas abordagens para escalabilidade e interoperabilidade. A pesquisa em provas de conhecimento zero (*zero-knowledge proofs*) tornou viáveis as soluções de camada 2, que executam transações fora da cadeia principal mantendo garantias de segurança herdadas da camada base Bashir (2023).

## 1.3 Sistemas Distribuídos: Fundamentos

### 1.3.1 Definição e Características

Para compreender blockchain em profundidade, é indispensável compreender os sistemas distribuídos, pois blockchain é, em sua essência, um sistema distribuído. Um sistema distribuído é um paradigma computacional no qual dois ou mais nós trabalham de maneira coordenada para alcançar um resultado comum, sendo modelado de tal forma que o usuário final o percebe como uma única plataforma lógica coerente Bashir (2023). Exemplos cotidianos incluem mecanismos de busca na internet e sistemas bancários, que operam sobre infraestruturas massivamente distribuídas invisíveis ao usuário.

Em um sistema distribuído, a comunicação entre nós ocorre por troca de mensagens por meio de canais de comunicação. Os nós podem ser honestos, defeituosos ou maliciosos. Um nó que exhibe comportamento arbitrário é denominado nó bizantino, em referência ao Problema dos Generais Bizantinos. Os dois desafios centrais do projeto de sistemas distribuídos são a coordenação entre nós e a tolerância a falhas: mesmo que parte dos nós ou dos enlaces de rede falhe, o sistema deve continuar operando de maneira correta Bashir (2023).

### 1.3.2 O Problema dos Generais Bizantinos

O Problema dos Generais Bizantinos, proposto por Lamport et al. em 1982, é um experimento mental que modela com precisão o desafio do consenso em ambientes não confiáveis. No problema, generais de um exército precisam coordenar um ataque simultâneo, comunicando-se apenas por mensageiros que podem ser interceptados ou substituídos por traidores. O desafio é chegar a um acordo válido mesmo na presença de generais traidores e mensagens perdidas.

Transposto para o domínio computacional: os generais são nós honestos; os traidores, nós com comportamento arbitrário (bizantino); os mensageiros representam os

canais de comunicação; e os mensageiros capturados correspondem a mensagens atrasadas ou perdidas. O Bitcoin, e o blockchain em geral, oferece uma solução prática para este problema histórico ao substituir a confiança mútua entre os participantes pelo custo verificável de produção de cada bloco, tornando o comportamento desonesto economicamente desvantajoso Bashir (2023).

### 1.3.3 O Teorema CAP

Um resultado fundamental da teoria de sistemas distribuídos é o Teorema CAP, originalmente proposto como conjectura por Eric Brewer em 1998 e demonstrado formalmente por Seth Gilbert e Nancy Lynch em 2002. O teorema afirma que nenhum sistema distribuído pode garantir simultaneamente as três propriedades a seguir Bashir (2023):

- **Consistência** (*Consistency*): todos os nós do sistema possuem, a qualquer instante, uma cópia idêntica e atualizada dos dados. Essa propriedade está diretamente ligada ao conceito de replicação de máquina de estados (*State Machine Replication*, SMR).
- **Disponibilidade** (*Availability*): todos os nós estão acessíveis, aceitando requisições e respondendo com dados sem falhas sempre que solicitados.
- **Tolerância a Partições** (*Partition tolerance*): o sistema continua operando corretamente mesmo quando um subconjunto de nós perde comunicação com os demais em decorrência de falhas de rede.

O teorema implica que, ocorrendo uma partição de rede, o projetista deve escolher entre consistência e disponibilidade. Blockchains públicas, como o Bitcoin, optam por priorizar disponibilidade e tolerância a partições, sacrificando a consistência imediata. Esse sacrifício é compensado pelo mecanismo de *consistência eventual*: ao longo do tempo, com a propagação e validação de novos blocos, todos os nós convergem para uma visão comum e consistente do histórico de transações. Em Bitcoin, múltiplas confirmações de bloco subsequentes elevam progressivamente a confiança de que uma transação não será revertida Bashir (2023).

### 1.3.4 O Teorema PACELC

Uma extensão do Teorema CAP, proposta por Daniel J. Abadi da Universidade de Yale, é denominada PACELC. Essa formulação ampliada reconhece que, mesmo na ausência de partições de rede, existe sempre uma tensão entre consistência e latência em sistemas replicados. Enquanto o CAP trata exclusivamente do comportamento do sistema durante partições, o PACELC estabelece que, sob operação normal, sistemas distribuídos devem escolher entre oferecer menor latência (possivelmente abrindo mão

de consistência imediata) ou manter consistência estrita (arcando com maior latência) Bashir (2023). Essa perspectiva é relevante para a análise de blockchains empresariais que, em ambientes controlados e sem partições frequentes, ainda enfrentam a dicotomia entre velocidade de finalização e garantias de consistência.

## 1.4 Arquitetura Geral de uma Blockchain

### 1.4.1 Definições Fundamentais

Blockchain pode ser definida de diferentes perspectivas. Em termos acessíveis, trata-se de um sistema de registro compartilhado, seguro e em constante crescimento, no qual cada participante mantém uma cópia dos registros, e estes somente podem ser atualizados quando a maioria dos participantes envolvidos em uma transação concorda com a atualização. Em termos técnicos mais precisos, blockchain é um ledger distribuído ponto-a-ponto, criptograficamente seguro, apenas de acréscimo (*append-only*), imutável na prática e atualizável exclusivamente por meio de consenso entre os pares Bashir (2023).

Cada um dos termos dessa definição técnica carrega peso conceitual significativo. *Ponto-a-ponto* indica a ausência de controlador central: todos os participantes comunicam-se diretamente entre si. *Ledger distribuído* significa que a base de dados está replicada entre todos os nós da rede. *Criptograficamente seguro* indica que serviços de segurança como irrefutabilidade, integridade de dados e autenticação de origem são garantidos por primitivas criptográficas. *Append-only* implica que dados só podem ser adicionados em ordem cronológica e que registros anteriores não podem ser alterados na prática. Por fim, a atualização via consenso confere à blockchain sua característica descentralizadora fundamental: nenhuma autoridade central controla o ledger Bashir (2023).

### 1.4.2 Visão em Camadas

A arquitetura de uma blockchain genérica pode ser compreendida por meio de um modelo em camadas hierárquicas Bashir (2023):

1. **Camada de Rede:** usualmente a internet, provê a base de comunicação para qualquer blockchain.
2. **Camada Ponto-a-Ponto:** protocolo de propagação de informações (como protocolos de *gossip* ou inundação) que opera sobre a camada de rede.
3. **Camada Criptográfica:** contém as primitivas criptográficas essenciais para a segurança da blockchain, incluindo criptografia de chave pública, assinaturas digitais e funções de hash criptográficas.

4. **Camada de Consenso:** responsável pela implementação dos mecanismos que garantem o acordo entre participantes sobre o estado final dos dados, podendo incluir mecanismos baseados em prova (PoW, PoS) ou protocolos de tolerância a falhas bizantinas tradicionais.
5. **Camada de Execução:** engloba máquinas virtuais, blocos, transações e contratos inteligentes. Nesta camada ocorrem operações como transferências de valor, execução de contratos e geração de blocos.
6. **Camada de Aplicações:** composta por contratos inteligentes, aplicações descentralizadas, DAOs e agentes autônomos. É por esta camada que os usuários interagem com a blockchain.

Essa analogia com a pilha TCP/IP, onde o blockchain funciona como um protocolo de camada de aplicação rodando sobre a internet, é instrutiva: assim como HTTP ou SMTP operam sobre TCP/IP, os protocolos de blockchain operam sobre a infraestrutura de rede existente Bashir (2023).

### 1.4.3 Elementos Genéricos

Uma blockchain é composta por um conjunto bem definido de elementos primitivos Bashir (2023):

**Endereços** (*Addresses*): identificadores únicos utilizados nas transações para denotar remetentes e destinatários. Um endereço é normalmente derivado de uma chave pública por meio de funções de hash.

**Transações** (*Transactions*): a unidade fundamental de uma blockchain. Uma transação representa a transferência de valor de um endereço para outro, podendo também representar a invocação de um contrato inteligente. A estrutura de dados de uma transação tipicamente contém a lógica de transferência, regras relevantes, endereços de origem e destino, e informações de validação.

**Blocos** (*Blocks*): agrupamentos de transações validadas, organizados logicamente em um cabeçalho (*block header*) e um conjunto de transações. O cabeçalho contém: (a) uma referência ao bloco anterior, representada pelo hash do cabeçalho desse bloco anterior, constituindo o encadeamento que dá nome à tecnologia; (b) um *nonce*, número usado uma única vez em algoritmos de PoW; (c) um *timestamp* com o momento de criação do bloco; (d) a *Merkle root*, o hash combinado de todas as transações do bloco, que permite verificar a integridade do conjunto completo de transações a partir de um único valor.

**Nós** (*Nodes*): participantes individuais na rede blockchain, responsáveis por propagar, validar e, em alguns casos, produzir novos blocos. Nós também executam a função de assinar transações digitalmente com chaves privadas, provando ser os legítimos proprietários dos ativos que desejam transferir.

**Mineradores e Validadores** (*Miners/Validators*): nós especializados que criam novos blocos e garantem a segurança do consenso. Em redes baseadas em PoW, os mineradores competem para resolver um quebra-cabeça computacional; em redes baseadas em PoS, os validadores são selecionados com base no capital em garantia (*stake*).

#### 1.4.4 O Processo de Consenso e Adição de Blocos

O processo pelo qual uma blockchain cresce pode ser descrito em cinco etapas Bashir (2023): (1) iniciação de uma transação por um nó, que a assina com sua chave privada; (2) propagação e validação da transação entre os pares da rede; (3) seleção da transação para inclusão em um bloco por um minerador ou validador; (4) finalização do bloco quando o nó produtor satisfaz o requisito do mecanismo de consenso vigente; (5) propagação do novo bloco pela rede, validação pelos demais nós e incorporação ao ledger, com o bloco seguinte apontando criptograficamente para este por meio de um ponteiro de hash.

## Capítulo 2

# Descentralização e Arquitetura de Rede

A descentralização é uma das ideias centrais da tecnologia blockchain. Neste capítulo, ela é tratada de modo direto: primeiro como diferença entre sistemas centralizados, distribuídos e descentralizados; depois como arquitetura de rede ponto-a-ponto; por fim, como critério para classificar diferentes tipos de blockchain. O objetivo é apresentar o tema de forma objetiva, sem transformar a monografia em uma taxonomia extensa demais.

### 2.1 O Conceito de Descentralização

A descentralização não é uma invenção da era blockchain. Em gestão organizacional, ciência política e teoria de sistemas, o termo descreve a distribuição de autoridade e controle para a periferia de uma estrutura, em oposição à sua concentração em um núcleo central. No entanto, foi com o advento do Bitcoin, em 2008, e da tecnologia blockchain que a descentralização ganhou uma expressão computacional precisa e verificável Bashir (2023).

No contexto de sistemas de informação, a descentralização pode ser definida como a ausência de um nó ou entidade central com autoridade exclusiva sobre o estado do sistema. Em uma rede descentralizada, nenhum participante isolado pode, por conta própria, alterar unilateralmente os registros, censurar transações ou impor regras sem o consentimento da maioria. Essa propriedade contrasta diretamente com os sistemas centralizados tradicionais, nos quais um único servidor ou autoridade detém controle absoluto sobre os dados e sobre quem pode acessá-los Bashir (2023).

A importância da descentralização manifesta-se em múltiplas dimensões. Do ponto de vista da segurança, a ausência de um ponto central de falha torna o sistema inerentemente mais resistente a ataques. Do ponto de vista da confiança, elimina a necessidade de que as partes se fiem em um intermediário comum, substituindo a confiança institucional pela confiança matemática e criptográfica. Do ponto de vista social e político, oferece uma infraestrutura que pode operar independentemente de jurisdições, censura ou intervenção de terceiros Bashir (2023).

É importante distinguir, ainda, entre sistemas distribuídos e sistemas descentralizados. Um sistema distribuído replica dados e computação por múltiplos nós, mas pode ainda assim possuir uma autoridade central coordenadora. Um sistema descentralizado, por sua vez, elimina precisamente essa autoridade central: os nós operam de forma autônoma e coordenada exclusivamente por protocolos de consenso. O blockchain é, em sua essência, um sistema distribuído e descentralizado simultaneamente Bashir (2023).

## 2.2 Benefícios da Descentralização

### 2.2.1 Resiliência e Eliminação do Ponto Único de Falha

Em arquiteturas centralizadas, a indisponibilidade do servidor central implica a interrupção total do serviço. Sistemas descentralizados, por replicarem o estado da rede em milhares de nós distribuídos geograficamente, eliminam esse ponto único de falha. Mesmo que uma fração significativa dos nós saia da rede, o sistema continua operacional, pois os nós restantes mantêm cópias íntegras do registro Bashir (2023).

### 2.2.2 Resistência à Censura

Em redes descentralizadas, não existe uma autoridade com poder de vetar ou reverter transações unilateralmente. Para que uma transação válida seja rejeitada, seria necessário que a maioria dos participantes da rede concordasse em fazê-lo, o que torna a censura economicamente custosa e operacionalmente complexa. Essa propriedade tem implicações relevantes em contextos de restrições políticas, controle de capitais ou supressão de dissidências Bashir (2023).

### 2.2.3 Transparência e Confiança Algorítmica

A transparência inerente às blockchains públicas permite que qualquer participante audite o histórico completo de transações. Em vez de confiar na honestidade de uma instituição, os participantes confiam na integridade matemática do protocolo. Essa substituição de confiança interpessoal por confiança computacional é uma das contribuições conceituais mais profundas da tecnologia blockchain Bashir (2023).

## 2.3 Rede Peer-to-Peer: Topologia e Comunicação

### 2.3.1 Topologia da Rede

O blockchain opera sobre uma rede *peer-to-peer* (P2P), na qual todos os participantes, denominados nós, podem comunicar-se diretamente entre si sem a mediação de



um servidor central. Nessa topologia, cada nó é simultaneamente cliente e servidor, capaz de tanto solicitar quanto fornecer dados e serviços Bashir (2023).

Os nós de uma rede blockchain podem assumir diferentes papéis. Nós completos (*full nodes*) mantêm uma cópia integral do histórico da blockchain, validam todas as transações e blocos de acordo com as regras do protocolo e propagam informações para outros nós. Nós leves (*lightweight nodes* ou *SPV nodes*, do inglês *Simplified Payment Verification*) armazenam apenas os cabeçalhos dos blocos e recorrem a nós completos para verificar transações específicas, sendo adequados para dispositivos com recursos computacionais limitados, como smartphones Bashir (2023).

### 2.3.2 Propagação de Dados e o Protocolo Gossip

A disseminação de informações em redes P2P blockchain frequentemente utiliza o protocolo *gossip* (também denominado protocolo epidêmico). Nesse modelo, quando um nó recebe uma nova transação ou bloco, o repassa para um subconjunto de seus pares, que por sua vez repetem o processo. Esse mecanismo de propagação em cascata garante que a informação alcance toda a rede em tempo logarítmico em relação ao número de nós, sem depender de qualquer coordenador central Bashir (2023).

Quando um nó cria ou recebe uma transação válida, essa transação é inicialmente armazenada em uma área de espera denominada *mempool* (*memory pool*). O *mempool* funciona como uma fila de transações pendentes aguardando confirmação, ou seja, inclusão em um bloco pela ação dos mineradores. A sincronização entre nós envolve não apenas a troca de transações do *mempool*, mas também o alinhamento do estado da blockchain, garantindo que todos os nós trabalhem com o mesmo histórico canônico Bashir (2023).

## 2.4 Descentralização em Prática: Contratos Inteligentes, Organizações e Agentes Autônomos

### 2.4.1 Contratos Inteligentes

Contratos inteligentes são programas de computador que residem e executam sobre uma blockchain. Eles encapsulam lógica de negócio que é ativada automaticamente quando condições predefinidas são satisfeitas, sem necessidade de intervenção humana para execução. Sua imutabilidade após implantação e sua execução determinística garantem que as partes envolvidas não possam contestar os resultados, desde que as condições sejam verificadas on-chain Bashir (2023).

## 2.4.2 Organizações Autônomas Descentralizadas

As Organizações Autônomas Descentralizadas (DAOs, do inglês *Decentralized Autonomous Organizations*) representam a expressão mais ambiciosa da descentralização: organizações inteiras codificadas como contratos inteligentes, operando sem hierarquia centralizada. As regras de governança, a alocação de recursos e os mecanismos de tomada de decisão são todos definidos no código e executados pela rede Bashir (2023).

O caso mais célebre e instrutivo de DAO foi o projeto homônimo lançado sobre o Ethereum em 2016, que captou aproximadamente 168 milhões de dólares em sua fase de arrecadação. Uma vulnerabilidade no código foi explorada por agentes maliciosos, resultando na drenagem de uma parcela substancial dos fundos. O episódio exigiu uma bifurcação (*hard fork*) da rede Ethereum para reverter as transações fraudulentas, gerando profundo debate sobre imutabilidade, governança e responsabilidade em sistemas descentralizados Bashir (2023).

## 2.4.3 Agentes Autônomos

Agentes Autônomos (AAs) são entidades de software, potencialmente dotadas de lógica de inteligência artificial, que atuam em nome de seus proprietários para atingir objetivos definidos com mínima ou nenhuma intervenção humana. Quando combinados com contratos inteligentes em blockchains, os agentes autônomos podem executar transações, responder a eventos externos e interagir com outros contratos de forma completamente automatizada, abrindo caminho para o conceito de *algocracy*: a governança mediada por algoritmos Bashir (2023).

## 2.5 O Dilema da Descentralização: Compromissos com Desempenho e Usabilidade

A descentralização não é um bem sem custo. Ela impõe compromissos (*trade-offs*) significativos que todo projetista de sistemas blockchain precisa considerar cuidadosamente.

O compromisso mais fundamental é com o desempenho. Em sistemas centralizados, a validação de uma transação requer apenas a aprovação de uma única entidade confiável, processo que pode ocorrer em milissegundos. Em sistemas descentralizados, a mesma validação requer consenso entre dezenas, centenas ou milhares de nós distribuídos globalmente, impondo latências e limitações de throughput que sistemas financeiros tradicionais como Visa ou Mastercard não enfrentam Bashir (2023).

A usabilidade também é afetada. A gestão de chaves privadas, a irreversibilidade das transações e a ausência de mecanismos de recuperação de acesso são barreiras con-

sideráveis para adoção em massa. Em sistemas centralizados, senhas esquecidas podem ser recuperadas; fundos transferidos erroneamente podem ser estornados. Em blockchains descentralizadas, a perda de uma chave privada pode significar a perda permanente de ativos, e transações errôneas são, em princípio, irreversíveis Bashir (2023).

Existe ainda o dilema regulatório: a descentralização, ao eliminar responsáveis identificáveis, dificulta a aplicação de normas de combate à lavagem de dinheiro, proteção ao consumidor e tributação. Esses desafios não invalidam a tecnologia, mas indicam que o nível ótimo de descentralização para um dado caso de uso raramente é o máximo tecnicamente possível Bashir (2023).

## 2.6 Tipos de Blockchain por Nível de Acesso

A evolução da tecnologia blockchain produziu uma taxonomia variada de redes, classificadas de acordo com os critérios de participação e controle.

### 2.6.1 Blockchains Públicas

Blockchains públicas são abertas à participação de qualquer indivíduo, sem necessidade de autorização prévia. Qualquer nó pode se juntar à rede, validar transações e contribuir com o processo de consenso. Bitcoin e Ethereum são os exemplos canônicos. Nessas redes, a transparência é total e o consenso é alcançado por mecanismos como Prova de Trabalho (*Proof of Work*) ou Prova de Participação (*Proof of Stake*), que desincentivam comportamentos desonestos mediante custos econômicos Bashir (2023).

### 2.6.2 Blockchains Privadas e de Consórcio

Blockchains privadas restringem a participação a um grupo predefinido de entidades. O acesso é controlado por uma autoridade central ou por mecanismo de convite. Essas redes são frequentemente denominadas *blockchains de consórcio* ou *blockchains corporativas*, sendo exemplos notáveis o Hyperledger Fabric e o Quorum. Nesses ambientes, a identidade dos participantes é conhecida e verificada, o que permite o uso de mecanismos de consenso mais eficientes que dispensam o alto custo computacional da mineração Bashir (2023).

### 2.6.3 Blockchains Semi-Privadas e Híbridas

Blockchains semi-privadas combinam elementos das arquiteturas pública e privada. Uma porção da rede permanece acessível publicamente, enquanto outra é restrita a participantes autorizados. Esse modelo híbrido atende cenários nos quais parte das informações

precisa ser auditável publicamente, enquanto dados sensíveis permanecem restritos a um subconjunto de participantes confiáveis Bashir (2023).

## 2.6.4 Ledgers Permissionados

Ledgers permissionados são aqueles em que os participantes são previamente conhecidos e autorizados. A verificação de transações é realizada por validadores pré-selecionados, eliminando a necessidade de mecanismos de consenso competitivos. Embora frequentemente confundidos com blockchains privadas, ledgers permissionados podem ser públicos em termos de visibilidade, mas com controle de acesso regulado para participação na validação Bashir (2023).

## 2.7 Tipos por Token: Blockchains Tokenizadas e Sem Token

A presença ou ausência de um ativo digital nativo subdivide as blockchains em duas categorias adicionais.

Blockchains tokenizadas geram uma criptomoeda como resultado natural do processo de consenso. O token serve simultaneamente como incentivo econômico para os validadores e como unidade de valor transferível na rede. Bitcoin e Ether (Ethereum) são os exemplos mais expressivos Bashir (2023).

Blockchains sem token (*tokenless blockchains*) operam sem uma criptomoeda nativa. São adequadas para cenários corporativos nos quais o objetivo é o compartilhamento seguro e imutável de dados entre partes confiáveis, sem necessidade de transferência de valor monetário. Hyperledger Fabric e Quorum são exemplos típicos. Nessas plataformas, tokens podem ser implementados como aplicações sobre a blockchain, mas não existem nativamente no protocolo base Bashir (2023).

## 2.8 Arquitetura em Camadas: Layer 1, Layer 2 e Além

### 2.8.1 Blockchains de Camada 1

Blockchains de Camada 1 (*Layer 1*) são redes base responsáveis pelo consenso, segurança e registro canônico das transações. Dentro dessa categoria, distinguem-se dois modelos arquiteturais principais Bashir (2023).

As arquiteturas monolíticas concentram todas as funcionalidades — consenso, execução de contratos inteligentes, disponibilidade de dados e segurança — em uma única cadeia base. Bitcoin, Ethereum e Solana são exemplos dessa abordagem. A vantagem é a simplicidade conceitual e a robustez do modelo; a desvantagem é a dificuldade em escalar

sem comprometer a descentralização ou a segurança, dilema frequentemente descrito como o *trilema da blockchain* Bashir (2023).

As arquiteturas polilíticas (ou multi-cadeia) distribuem as responsabilidades entre múltiplas cadeias interconectadas. Polkadot, Avalanche e Cosmos são exemplos paradigmáticos. Nessas arquiteturas, uma cadeia central (*relay chain* ou equivalente) coordena a segurança e a interoperabilidade, enquanto cadeias secundárias (*parachains*, *subnets* ou *zones*) executam transações especializadas. Quando as cadeias secundárias seguem as mesmas regras, denominam-se homogêneas; quando seguem regras distintas, heterogêneas Bashir (2023).

## 2.8.2 Blockchains de Camada 2

Soluções de Camada 2 (*Layer 2*) emergem como resposta às limitações de escalabilidade das cadeias base. Em vez de competir com a Camada 1, estendem-na: executam transações fora da cadeia principal (*off-chain*), recorrendo à Camada 1 apenas para liquidação final e garantia de segurança Bashir (2023).

Entre as principais soluções de Camada 2 destacam-se os *rollups* otimistas, que presumem a validade das transações e abrem janelas para contestação; os *rollups* de conhecimento zero (*zero-knowledge rollups*), que utilizam provas criptográficas para verificar a validade das transações sem revelar seus dados; as *sidechains*, como Rootstock, que estendem funcionalidades do Bitcoin por meio de mecanismos de *peg* bidirecional; os *plasma chains*; e a *Lightning Network*, solução de canais de pagamento para o Bitcoin. O denominador comum dessas soluções é a busca pelo aumento de throughput e redução de latência sem sacrificar as garantias de segurança da cadeia base Bashir (2023).

# Capítulo 3

## Fundamentos Criptográficos

A segurança das redes blockchain repousa sobre um conjunto coeso de técnicas matemáticas denominadas criptografia. Sem esses mecanismos, não seria possível garantir a autenticidade das transações, a imutabilidade dos registros ou a identidade dos participantes. Este capítulo apresenta os fundamentos criptográficos indispensáveis para compreender como blockchains protegem dados e asseguram confiança em ambientes distribuídos e sem autoridade central Bashir (2023).

### 3.1 Introdução à Criptografia

Criptografia é a ciência de tornar a informação segura na presença de adversários. Cifras são algoritmos empregados para cifrar ou decifrar dados de modo que, se interceptados por um adversário, sejam ininteligíveis sem a posse de uma chave secreta. Por si só, a criptografia não constitui uma solução completa de segurança; ela atua como bloco fundamental dentro de sistemas de segurança mais amplos. A proteção de um ecossistema blockchain, por exemplo, exige a combinação de diversas primitivas criptográficas: funções de hash, criptografia de chave simétrica, assinaturas digitais e criptografia de chave pública Bashir (2023).

#### 3.1.1 Serviços Providos pela Criptografia

Os serviços de segurança fornecidos pela criptografia podem ser organizados em quatro categorias principais.

**Confidencialidade** é a garantia de que a informação está disponível apenas para entidades autorizadas. É o serviço mais imediatamente associado à criptografia e é obtido por meio da cifragem dos dados com chaves secretas ou públicas.

**Integridade** é a garantia de que a informação somente pode ser modificada por entidades autorizadas. Em blockchains, a integridade dos blocos e das transações é verificada por meio de funções de hash, que tornam qualquer alteração computacionalmente

detectável.

**Autenticação** provê garantias sobre a identidade de uma entidade ou sobre a validade de uma mensagem. Divide-se em dois subtipos: autenticação de entidade, que assegura que um participante está ativamente envolvido em uma sessão de comunicação; e autenticação de origem dos dados, que verifica que a fonte da informação é genuína. Esta última garante também integridade, pois, se a origem é verificada, os dados não foram alterados. Métodos como códigos de autenticação de mensagem (MACs) e assinaturas digitais são empregados para esse fim Bashir (2023).

**Não-repúdio** é a garantia de que uma entidade não pode negar um compromisso ou ação previamente realizada. Essa propriedade é essencial em transações comerciais e financeiras, inclusive nas blockchains, onde uma parte que assinou e transmitiu uma transação não pode, posteriormente, alegar que não o fez. O não-repúdio é provido por assinaturas digitais, que utilizam a chave privada do signatário como evidência criptográfica incontestável Bashir (2023).

Adicionalmente, a **responsabilização** (accountability) assegura que ações que afetam a segurança possam ser rastreadas até a parte responsável, geralmente por meio de registros e mecanismos de auditoria.

## 3.2 Primitivas Criptográficas

Primitivas criptográficas são os blocos de construção básicos de protocolos e sistemas de segurança. Dividem-se em três grandes categorias: primitivas sem chave, primitivas de chave simétrica e primitivas de chave assimétrica Bashir (2023).

### 3.2.1 Primitivas Sem Chave

As primitivas sem chave incluem geradores de números aleatórios e funções de hash. Os geradores de números aleatórios (RNGs) fazem uso da aleatoriedade disponível no mundo real, como variações de temperatura, ruídos térmicos ou movimentos do cursor. Geradores pseudoaleatórios (PRNGs), por sua vez, são funções determinísticas que utilizam um valor inicial denominado semente para produzir sequências de aparência aleatória. Um exemplo clássico é o algoritmo Blum-Blum-Shub (BBS). PRNGs são preferíveis a RNGs em aplicações criptográficas por sua confiabilidade e natureza determinística Bashir (2023).

### 3.2.2 Primitivas de Chave Simétrica

As primitivas de chave simétrica incluem cifras de chave secreta (cifras de fluxo e cifras de bloco) e códigos de autenticação de mensagem (MACs). Em todos esses esquemas, a mesma chave é utilizada para cifrar e decifrar os dados. A chave deve ser

estabelecida entre as partes antes do início da troca de informações, o que representa um desafio logístico mitigado por protocolos de acordo de chaves, como o protocolo Diffie-Hellman.

### 3.2.3 Primitivas de Chave Assimétrica

As primitivas assimétricas utilizam um par de chaves matematicamente relacionadas: uma chave pública, distribuída livremente, e uma chave privada, mantida em sigilo. Incluem cifras de chave pública e esquemas de assinatura digital. Embora computacionalmente mais lentas que as simétricas, as primitivas assimétricas resolvem o problema de distribuição de chaves e proveem não-repúdio Bashir (2023).

## 3.3 Funções de Hash

Funções de hash são funções sem chave que produzem um resumo de comprimento fixo a partir de uma entrada de tamanho arbitrário. São funções de sentido único e proveem o serviço de integridade de dados Bashir (2023).

### 3.3.1 Propriedades das Funções de Hash

As propriedades práticas das funções de hash incluem a capacidade de comprimir mensagens de tamanho arbitrário em resumos de comprimento fixo, tipicamente entre 128 e 512 bits, e a eficiência computacional.

As propriedades de segurança são três:

**Resistência à pré-imagem** (one-way): dado um valor  $y$ , é computacionalmente inviável encontrar  $x$  tal que  $h(x) = y$ . Essa propriedade garante que o dado original não pode ser recuperado a partir do resumo.

**Resistência à segunda pré-imagem** (resistência fraca a colisões): dado  $x$ , é computacionalmente inviável encontrar  $x' \neq x$  tal que  $h(x') = h(x)$ .

**Resistência a colisões** (resistência forte a colisões): é computacionalmente inviável encontrar quaisquer dois valores distintos  $x'$  e  $x$  tais que  $h(x') = h(x)$  Bashir (2023).

Uma propriedade adicional desejável é o **efeito avalanche**: uma pequena alteração na entrada, até mesmo de um único caractere, produz uma saída completamente diferente. Esse comportamento impede que adversários deduzam informações sobre a entrada com base em pequenas variações do resumo.

### 3.3.2 SHA-256 e Aplicações em Blockchain

O SHA-256 pertence à família SHA-2, padronizada pelo NIST. Possui limite de entrada de  $2^{64} - 1$  bits, tamanho de bloco de 512 bits, palavra de 32 bits e saída de 256



bits. É uma construção de Merkle-Damgård: a mensagem é dividida em blocos de 512 bits processados iterativamente por uma função de compressão. O algoritmo opera em 64 rodadas por bloco, utilizando funções auxiliares como  $\text{Maj}(a, b, c) = (a \wedge b) \oplus (a \wedge c) \oplus (b \wedge c)$  e  $\text{Ch}(e, f, g) = (e \wedge f) \oplus (\neg e \wedge g)$ , além de rotações de bits Bashir (2023).

Bitcoin utiliza SHA-256 em seu mecanismo de prova de trabalho (proof of work), no qual os mineradores devem encontrar um valor de nonce tal que o hash do cabeçalho do bloco seja inferior a um alvo determinado. Ethereum, por sua vez, utiliza o Keccak-256, versão original do algoritmo Keccak apresentado ao NIST antes das modificações que resultaram no padrão SHA-3 Bashir (2023).

As funções de hash têm múltiplas aplicações em blockchains: geração de endereços a partir de chaves públicas; encadeamento de blocos, em que o cabeçalho de cada bloco contém o hash do bloco anterior; construção de árvores de Merkle para garantir a integridade das transações; e mecanismos de consenso baseados em prova de trabalho.

### 3.3.3 Árvore de Merkle

Introduzida por Ralph Merkle, a árvore de Merkle é uma estrutura de dados em forma de árvore binária em que os dados de entrada são posicionados nas folhas e os valores dos nós internos são produzidos pelo hash concatenado dos valores dos filhos, até que um único valor — a raiz de Merkle — seja obtido. Qualquer alteração em qualquer dado folha modifica a raiz de Merkle, permitindo verificação eficiente de integridade Bashir (2023).

Em blockchains, a raiz de Merkle de todas as transações de um bloco é incluída no cabeçalho do bloco. Isso permite que nós leves (clientes SPV em Bitcoin) verifiquem a inclusão de uma transação sem baixar o bloco completo, utilizando apenas uma prova de Merkle — o conjunto mínimo de hashes intermediários necessários para reconstruir a raiz Bashir (2023).

## 3.4 Criptografia Simétrica

A criptografia simétrica, também denominada criptografia de chave secreta ou compartilhada, utiliza a mesma chave para cifrar e decifrar os dados. É computacionalmente eficiente e adequada para cifrar grandes volumes de dados Bashir (2023).

No contexto de blockchain, a criptografia simétrica aparece como mecanismo auxiliar. Ela pode proteger chaves privadas armazenadas em carteiras ou dados sensíveis em sistemas de apoio, mas não resolve sozinha o problema central de uma blockchain pública: provar propriedade e autorização em uma rede aberta, sem compartilhar segredo com todos os participantes. Para isso, a tecnologia depende principalmente de criptografia assimétrica e assinaturas digitais.

## 3.5 Criptografia Assimétrica

A criptografia assimétrica, também chamada de criptografia de chave pública, utiliza um par de chaves matematicamente relacionadas: uma chave pública, compartilhada livremente, e uma chave privada, conhecida apenas pelo seu detentor. A relação matemática entre as chaves é fundamentada em problemas computacionalmente difíceis de resolver na direção inversa Bashir (2023).

### 3.5.1 Fundamentos Matemáticos

Três classes de problemas difíceis sustentam os principais algoritmos assimétricos.

A **fatoração de inteiros** baseia-se no fato de que, embora seja trivial multiplicar dois números primos grandes, fatorar o produto resultante é computacionalmente inviável. O RSA fundamenta-se nessa propriedade.

O **logaritmo discreto** em aritmética modular explora a dificuldade de, dado  $g^x \bmod p = y$ , encontrar  $x$  a partir de  $y$ ,  $g$  e  $p$ . O protocolo Diffie-Hellman e o algoritmo DSA são exemplos de aplicações desse problema.

O **logaritmo discreto em curvas elípticas** é especialmente relevante para blockchains como Bitcoin e Ethereum. Em termos práticos, uma chave privada é um número secreto; a chave pública é derivada desse número por uma operação matemática fácil de calcular, mas impraticável de inverter. Essa relação permite que qualquer nó verifique uma assinatura sem conhecer a chave privada do usuário.

## 3.6 Chaves Pública e Privada: Geração e Segurança

A chave privada é um número gerado aleatoriamente, mantido em segredo absoluto por seu detentor. Em sistemas baseados em ECC, como Bitcoin e Ethereum, a chave privada é um inteiro de 256 bits gerado por um PRNG criptograficamente seguro. A chave pública é derivada da privada por meio da multiplicação escalar na curva elíptica:  $K_{pub} = d \cdot G$  Bashir (2023).

A segurança do esquema depende inteiramente do sigilo da chave privada e da dificuldade computacional do ECDLP. Se a chave privada for comprometida, todo o esquema é invalidado. Por esse motivo, carteiras de criptomoedas empregam esquemas robustos de proteção, como a cifragem da chave privada com AES derivado de uma senha via Scrypt (Ethereum) ou AES-256-CBC (Bitcoin) Bashir (2023).

Chaves podem ser efêmeras, utilizadas em uma única sessão, ou estáticas, de longo prazo. Em protocolos de troca de chaves como ECDH, chaves efêmeras são preferidas para garantir sigilo de encaminhamento (forward secrecy).

### 3.7 Assinaturas Digitais

Assinaturas digitais associam uma mensagem à chave privada que a autorizou, provendo autenticação de origem e não-repúdio. Em blockchains, esse mecanismo é usado para provar que uma transação foi autorizada pelo detentor da chave privada correspondente ao endereço de origem Bashir (2023).

O fluxo conceitual é simples: o usuário cria uma transação, calcula-se um resumo criptográfico dessa transação e a carteira assina esse resumo com a chave privada. Os nós da rede verificam a assinatura com dados públicos. Se a assinatura for válida e as demais regras do protocolo forem satisfeitas, a transação pode ser aceita. Assim, a rede não precisa conhecer a identidade civil do usuário; basta verificar matematicamente que ele controla a chave correta.

### 3.8 Endereços em Blockchain: Derivação da Chave Pública via Hash

Endereços em blockchains são derivados da chave pública por meio de uma sequência de funções de hash, garantindo compacidade e uma camada adicional de segurança.

Em **Bitcoin**, o endereço é obtido pelos seguintes passos: (1) gera-se a chave pública ECDSA a partir da chave privada; (2) aplica-se SHA-256 sobre a chave pública; (3) aplica-se RIPEMD-160 sobre o resultado, produzindo um resumo de 160 bits; (4) adiciona-se um byte de versão; (5) calcula-se um checksum de 4 bytes (duplo SHA-256); (6) codifica-se o resultado em Base58Check, um esquema que evita caracteres ambíguos como 1, 0, O e I, prevenindo erros de transcrição Bashir (2023).

Em **Ethereum**, o endereço é obtido aplicando-se Keccak-256 diretamente sobre a chave pública (sem o byte prefixo de 04) e retendo os últimos 20 bytes (40 caracteres hexadecimais) do resumo resultante. O endereço é representado em hexadecimal prefixado por 0x Bashir (2023).

Essa derivação é uma função de sentido único: dado um endereço, é computacionalmente inviável recuperar a chave pública ou privada correspondente. Além disso, o uso de RIPEMD-160 em Bitcoin reduz o tamanho do endereço e provê uma segunda camada de proteção, de modo que mesmo que vulnerabilidades futuras sejam descobertas no ECDSA, o endereço não revelaria diretamente a chave pública enquanto nenhuma transação de saída for realizada.

# Capítulo 4

## Algoritmos de Consenso

O funcionamento de qualquer rede blockchain depende de um mecanismo fundamental: a capacidade de múltiplos participantes, sem autoridade central, chegarem a um acordo sobre o estado compartilhado do sistema. Este mecanismo é denominado *consenso distribuído*, e sua concepção é, ao mesmo tempo, um dos maiores desafios e uma das maiores conquistas da ciência da computação moderna. Neste capítulo, examinamos os fundamentos teóricos do problema do consenso, os principais algoritmos empregados em redes blockchain, e os trade-offs inerentes a cada abordagem Bashir (2023).

### 4.1 O Problema do Consenso em Sistemas Distribuídos

Em um sistema distribuído, processos independentes comunicam-se exclusivamente por troca de mensagens. Não existe memória compartilhada, relógio global ou autoridade coordenadora. Nesse cenário, obter consenso significa garantir que todos os processos corretos concordem com um único valor, mesmo diante de falhas e atrasos na rede.

O estudo formal do problema remonta ao final da década de 1970. Uma das contribuições mais relevantes é o resultado de impossibilidade FLP, publicado em 1985 por Fischer, Lynch e Paterson. O teorema demonstra que, em um sistema completamente assíncrono — onde não há limite superior conhecido para atrasos de mensagens —, é impossível garantir consenso determinístico mesmo que apenas um único processo falhe Bashir (2023). Este resultado não é meramente teórico: ele delimita o espaço de soluções práticas e obriga os projetistas de sistemas a escolherem entre relaxar as premissas de assincronia, introduzir mecanismos probabilísticos ou aceitar que o consenso pode não terminar em tempo finito.

Para contornar a impossibilidade FLP, três abordagens principais são utilizadas na prática: detectores de falhas (implementados tipicamente como timeouts), algoritmos randomizados que oferecem terminação probabilística, e suposições de sincronismo parcial, nas quais o sistema eventualmente se comporta de forma síncrona após um instante desconhecido denominado *Global Stabilization Time* (GST).

### 4.1.1 Requisitos Fundamentais

Todo algoritmo de consenso deve satisfazer duas categorias de propriedades Bashir (2023):

- **Segurança** (*Safety*): nada de ruim acontece. Inclui as propriedades de *acordo* (nenhum par de processos corretos decide valores diferentes), *validade* (o valor decidido deve ter sido proposto por algum processo honesto) e *integridade* (cada processo decide no máximo uma vez).
- **Vivacidade** (*Liveness*): algo de bom eventualmente acontece. A propriedade de *terminação* exige que cada processo correto tome uma decisão em tempo finito.

Existe uma tensão fundamental entre segurança e vivacidade em sistemas assíncronos, precisamente o que o resultado FLP captura. Algoritmos práticos geralmente privilegiam a segurança e estabelecem condições de sincronismo para garantir a vivacidade.

### 4.1.2 Tolerância a Falhas

As falhas em sistemas distribuídos são classificadas em dois grandes grupos Bashir (2023):

- **Falhas de colapso** (*crash faults*): um processo para de responder, mas não envia mensagens incorretas. Algoritmos tolerantes a esse tipo de falha são chamados de *Crash Fault-Tolerant* (CFT). Para tolerar  $f$  falhas, são necessários pelo menos  $2f + 1$  nós.
- **Falhas Bizantinas**: um processo pode se comportar de maneira arbitrária e maliciosa, enviando mensagens contraditórias ou incorretas para diferentes participantes. Algoritmos tolerantes a esse tipo são *Byzantine Fault-Tolerant* (BFT), e exigem pelo menos  $3f + 1$  nós para tolerar  $f$  falhas.

A replicação de máquina de estados (*State Machine Replication*, SMR) é a técnica canônica para obter tolerância a falhas em sistemas distribuídos. A ideia central é que todos os servidores iniciem com o mesmo estado, recebam requisições na mesma ordem total e executem as mesmas transições determinísticas. Consequentemente, todos mantêm estados idênticos. A difusão com ordem total (*total order broadcast*) e o consenso distribuído são, sob essa perspectiva, problemas equivalentes Bashir (2023).

## 4.2 Falhas Bizantinas e o Teorema dos Generais Bizantinos

O conceito de falha Bizantina foi formalizado por Lamport, Shostak e Pease em 1982, por meio do célebre *Problema dos Generais Bizantinos*. No problema, um conjunto de generais do exército bizantino, situados ao redor de uma cidade inimiga, precisa coordenar um ataque simultâneo. A comunicação é feita exclusivamente por mensageiros. Alguns generais podem ser traidores e enviar mensagens contraditórias para subverter o plano. A questão é: existe um algoritmo pelo qual os generais leais sempre chegam ao mesmo plano de ação?

A resposta é sim, desde que o número de traidores  $f$  seja estritamente menor que um terço do total  $n$  de generais, ou seja,  $n \geq 3f + 1$ . Esse resultado tem implicação direta para blockchains públicas: em redes onde participantes podem agir maliciosamente — seja por interesse econômico, ataque coordenado ou simples erro de implementação —, é necessário que a maioria qualificada de dois terços dos participantes seja honesta.

Em blockchains, essa ameaça se materializa de diversas formas: ataques de 51% em redes PoW, ataques de duplo gasto, e comportamentos equívocos (*equivocation*), nos quais um validador assina blocos conflitantes. Algoritmos BFT são projetados especificamente para resistir a essas ameaças.

## 4.3 Proof of Work

O mecanismo de *Proof of Work* (PoW) foi introduzido com o Bitcoin em 2009 e representa a primeira solução prática e escalável ao problema do consenso em redes públicas e sem permissão (*permissionless*). É importante notar que, contrariamente à percepção comum, o PoW não é, estritamente falando, um algoritmo de consenso: trata-se de um mecanismo de defesa contra ataques Sybil. O consenso efetivo é obtido pela aplicação da regra de escolha de cadeia (*fork choice rule*), que seleciona a cadeia mais longa ou mais pesada como a cadeia canônica Bashir (2023).

### 4.3.1 Mecanismo e Busca do Nonce

No PoW, o nó que deseja propor um bloco deve resolver um puzzle criptográfico. A condição a ser satisfeita é:

$$H(N \parallel P_{\text{hash}} \parallel Tx_1 \parallel \cdots \parallel Tx_k) < \text{Target}$$

onde  $H$  é uma função de hash criptográfica (SHA-256 no Bitcoin),  $N$  é o *nonce*,  $P_{\text{hash}}$  é o hash do bloco anterior e  $Tx_i$  são as transações incluídas no bloco Bashir (2023).

O único método viável para encontrar  $N$  é a força bruta: o minerador incrementa o nonce e recalcula o hash repetidamente até que a condição seja satisfeita.

### 4.3.2 Target Hash e Dificuldade Ajustável

O valor *Target* determina a dificuldade do puzzle. Quanto menor o *Target*, maior o número de zeros exigidos no início do hash resultante, e maior a dificuldade. No Bitcoin, a dificuldade é recalibrada a cada 2.016 blocos — aproximadamente duas semanas — de modo a manter o intervalo médio de geração de blocos próximo a dez minutos. Se o poder computacional agregado da rede aumentou desde o ajuste anterior, a dificuldade sobe; se diminuiu, a dificuldade cai Bashir (2023).

### 4.3.3 Segurança por Custo Computacional

A segurança do PoW repousa sobre um princípio econômico: para reescrever a história da blockchain, um atacante precisaria refazer todo o trabalho computacional da cadeia honesta, o que é economicamente inviável enquanto os participantes honestos controlarem a maioria do poder de mineração. Essa propriedade é frequentemente denominada *segurança por custo computacional*: atacar a rede é mais caro do que participar honestamente.

No entanto, o PoW apresenta limitações relevantes. O consumo energético é colossal — estimativas apontam que a rede Bitcoin consome mais eletricidade anualmente do que vários países. Além disso, a proliferação de ASICs (*Application-Specific Integrated Circuits*) concentrou o poder de mineração em grandes *pools*, ameaçando a descentralização. Variantes resistentes a ASICs, como os algoritmos de PoW ligados à memória (*memory-bound*), como o Equihash, tentam mitigar esse problema ao tornar o desempenho dependente da velocidade e do tamanho da RAM, e não apenas da capacidade de cálculo Bashir (2023).

## 4.4 Proof of Stake

O *Proof of Stake* (PoS), também chamado de mineração virtual, foi proposto pela primeira vez na criptomoeda Peercoin em 2012. Redes de grande relevância como Ethereum 2.0, Cardano, Polkadot e Tezos adotam variantes do PoS Bashir (2023).

### 4.4.1 Validadores e Stake como Garantia

No PoS, o direito de propor e validar blocos está condicionado ao depósito de uma quantidade de criptomoeda como garantia — o *stake*. A lógica subjacente é simples: um participante que tem capital comprometido na rede tem um incentivo econômico

direto para agir honestamente, pois um comportamento malicioso pode resultar na perda parcial ou total do stake depositado, mecanismo denominado *slashing*. A dificuldade de acumular grandes quantidades de criptomoeda é apresentada como barreira equivalente ao custo computacional do PoW.

#### 4.4.2 Seleção de Validadores

A seleção do próximo proponente de bloco é, em geral, feita de forma aleatória e ponderada pelo stake. Diversas abordagens existem Bashir (2023):

- **PoS baseado em idade da moeda** (*coin age*): a probabilidade de seleção é proporcional ao produto da quantidade de moedas pelo tempo desde a última utilização.
- **PoS baseado em comitê**: um grupo de validadores é escolhido aleatoriamente, tipicamente via *Verifiable Random Function* (VRF), e os membros propõem e votam em blocos em sequência. Esse modelo é utilizado no protocolo Ouroboros, do Cardano.
- **PoS delegado** (DPoS): os detentores de stake votam em um conjunto fixo de delegados, que produzem blocos em rodízio. Utilizado em EOS, Lisk e Cosmos. Embora eficiente, o DPoS tende à concentração de poder em um pequeno número de delegados conhecidos, comprometendo a descentralização.

#### 4.4.3 Slashing e Recompensas

O mecanismo de *slashing* penaliza validadores que se comportam de maneira equívoca — por exemplo, assinar dois blocos conflitantes na mesma altura de bloco. A penalidade é proporcional à gravidade da infração e pode envolver a destruição de parte ou totalidade do stake. As recompensas, por outro lado, são distribuídas na forma de taxas de transação e emissão de novos tokens, incentivando a participação honesta.

#### 4.4.4 O Problema do Nothing at Stake

Uma vulnerabilidade teórica do PoS é o problema *nothing at stake*: como não há custo computacional para minerar em múltiplas cadeias simultâneas, um validador racional pode apostar em todas as bifurcações (*forks*), maximizando recompensas independentemente de qual cadeia prevaleça. Isso facilita ataques de duplo gasto. O mecanismo de slashing é a principal contramedida adotada para eliminar esse incentivo Bashir (2023).



## 4.5 Comparação entre PoW e PoS

Tabela 4.1: Comparação entre Proof of Work e Proof of Stake

| <b>Critério</b>             | <b>PoW</b>               | <b>PoS</b>                        |
|-----------------------------|--------------------------|-----------------------------------|
| Consumo energético          | Muito alto               | Baixo                             |
| Risco de centralização      | ASICs e pools            | Concentração de stake             |
| Segurança                   | Custo computacional      | Custo econômico (slashing)        |
| Finalidade                  | Probabilística           | Determinística (em variantes BFT) |
| Velocidade de transação     | Baixa                    | Alta                              |
| Resistência a ataques Sybil | Alta (custo de hardware) | Alta (custo de stake)             |

O PoW oferece a propriedade de finalidade probabilística: a confiança de que uma transação é irreversível cresce com o número de blocos subsequentes. Na rede Bitcoin, convencionou-se aguardar seis confirmações — equivalentes a aproximadamente uma hora — para considerar uma transação definitivamente confirmada Bashir (2023). O PoS, quando combinado com protocolos BFT, pode oferecer finalidade determinística: uma vez que o bloco é finalizado, não há possibilidade de reorganização.

## 4.6 Trade-offs de Consenso

### 4.6.1 O Teorema CAP Revisitado

O Teorema CAP, enunciado por Eric Brewer em 2000 e formalmente provado por Gilbert e Lynch em 2002, estabelece que um sistema distribuído não pode garantir simultaneamente as três propriedades a seguir: *Consistency* (consistência), *Availability* (disponibilidade) e *Partition tolerance* (tolerância a partições). Em face de uma partição de rede, o sistema precisa escolher entre manter consistência (recusando responder até que a partição seja resolvida) ou disponibilidade (respondendo com dados potencialmente desatualizados).

Sistemas blockchain baseados em BFT, como PBFT e Tendermint, privilegiam a consistência: um bloco só é finalizado quando há quórum suficiente de confirmações, o que significa que, em caso de partição, o sistema pode parar de produzir blocos. Sistemas baseados em PoW privilegiam a disponibilidade: a cadeia continua crescendo mesmo sob partição, eventualmente convergindo por meio da regra da cadeia mais longa, ao custo de reorganizações temporárias.

## 4.6.2 O Triângulo Impossível: Latência, Segurança e Descentralização

No ecossistema blockchain, emergiu um corolário prático do teorema CAP frequentemente denominado *triângulo impossível* ou *trilema blockchain*: é extremamente difícil otimizar simultaneamente segurança, escalabilidade e descentralização Bashir (2023). As implicações são as seguintes:

- **Segurança vs. descentralização:** aumentar o número de validadores melhora a descentralização, mas eleva a complexidade de comunicação e reduz o desempenho.
- **Segurança vs. latência:** garantias mais fortes de finalidade exigem mais rodadas de votação, aumentando a latência por transação.
- **Latência vs. descentralização:** reduzir a latência frequentemente requer limitar o conjunto de validadores, caminhando em direção à centralização.

Redes como Ethereum optam por soluções de segunda camada (*Layer 2*) para escalar sem sacrificar a segurança da camada base. Redes permissionadas, ao restringir o conjunto de validadores, obtêm alto desempenho e finalidade determinística, mas abdicam da abertura e da resistência à censura características das redes públicas.

## 4.6.3 Finalidade: Probabilística versus Determinística

A finalidade de uma transação é uma propriedade crucial para aplicações financeiras e contratos críticos Bashir (2023). A finalidade probabilística do PoW, embora suficiente para muitos contextos, impõe latências da ordem de minutos a horas em redes como Bitcoin. A finalidade determinística dos algoritmos BFT, por sua vez, é obtida ao custo de escalabilidade e da exigência de um conjunto de validadores conhecido a priori, o que não é compatível com o modelo de redes abertas e sem permissão.

# Capítulo 5

## Bitcoin — A Pioneira

O Bitcoin representa muito mais do que a primeira criptomoeda bem-sucedida: ele constitui a prova de conceito que demonstrou ser possível construir um sistema monetário digital, descentralizado e resistente à censura sem a necessidade de qualquer autoridade central. Neste capítulo, examina-se a arquitetura técnica do Bitcoin, partindo do contexto histórico que motivou sua criação e percorrendo cada camada do sistema — criptografia de chaves, estrutura de blocos, modelo de transações UTXO, mecanismo de consenso por Prova de Trabalho, rede P2P e gestão de carteiras — até alcançar uma avaliação crítica de suas limitações fundamentais Bashir (2023).

### 5.1 Contexto Histórico e o Whitepaper de Satoshi Nakamoto

Em outubro de 2008, em plena crise financeira global deflagrada pelo colapso do mercado de hipotecas subprime norte-americano, um documento intitulado *Bitcoin: A Peer-to-Peer Electronic Cash System* foi publicado em uma lista de discussão de criptografia por um autor identificado como Satoshi Nakamoto Bashir (2023). A identidade real por trás desse pseudônimo permanece desconhecida até hoje e é objeto de ampla especulação acadêmica e jornalística. O momento não foi acidental: a crise evidenciara de maneira dramática os riscos sistêmicos de um sistema financeiro centralizado, dependente de instituições bancárias consideradas “grandes demais para falir” e sujeito a resgates custeados pelo contribuinte. A mensagem embutida no bloco gênese do Bitcoin — “*The Times 03/Jan/2009 Chancellor on brink of second bailout for banks*” — é um comentário deliberado sobre esse cenário.

O whitepaper apresentou uma solução elegante para o problema do duplo gasto (*double-spending*), que historicamente impedira a criação de dinheiro digital sem um intermediário confiável. O duplo gasto ocorre quando um mesmo ativo digital é enviado simultaneamente a dois destinatários distintos; sem uma autoridade central para arbitrar,

não há como determinar qual transação é legítima. O Bitcoin resolve isso por meio de um ledger distribuído e imutável — a blockchain — combinado a um mecanismo de validação e confirmação de transações baseado em consenso descentralizado. Em janeiro de 2009, a rede Bitcoin foi ao ar com a mineração do bloco gênese, inaugurando uma nova era para os sistemas monetários digitais.

Nos anos seguintes, o Bitcoin despertou interesse crescente tanto na academia quanto na indústria, abrindo linhas de pesquisa sobre sistemas distribuídos, criptografia aplicada, economia de tokens e governança descentralizada. Hoje é negociado como instrumento financeiro em bolsas regulamentadas, incluindo o contrato futuro na Chicago Mercantile Exchange (CME), e possui bilhões de dólares em capitalização de mercado Bashir (2023).

## 5.2 Criptografia de Chaves e Posse de Bitcoin

A propriedade e a transferência de valor no Bitcoin fundamentam-se em criptografia assimétrica. Três conceitos encadeados definem a “posse” de bitcoins: a chave privada, a chave pública e o endereço Bashir (2023).

### 5.2.1 Chave Privada

A chave privada é um número de 256 bits escolhido aleatoriamente dentro do intervalo definido pela curva elíptica SECP256K1, utilizada pelo algoritmo ECDSA (*Elliptic Curve Digital Signature Algorithm*). Qualquer valor entre `0x1` e `0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEBAAEDCE6AF48A03BBFD25E8CD0364140` constitui uma chave privada válida Bashir (2023). A segurança de todo o sistema repousa sobre o fato de que, embora seja computacionalmente trivial derivar a chave pública a partir da privada, a operação inversa é computacionalmente inviável dado o estado atual da computação clássica.

Para facilitar o armazenamento e a cópia, as chaves privadas são frequentemente codificadas no formato WIF (*Wallet Import Format*), que as converte em uma string alfanumérica mais compacta. Existe também o formato de chave privada mini, com no máximo 30 caracteres, empregado em contextos onde o espaço físico é escasso, como moedas físicas de Bitcoin (por exemplo, as Casascius coins) ou QR codes com alta tolerância a danos.

A chave privada deve permanecer em sigilo absoluto: quem a detém possui controle irrestrito sobre os bitcoins associados ao endereço correspondente. Não há mecanismo de recuperação em caso de perda, e não há autoridade capaz de reverter uma transação realizada com uma chave comprometida.

### 5.2.2 Chave Pública

A chave pública é derivada da chave privada mediante operações sobre a curva elíptica SECP256K1. É composta pelas coordenadas  $(x, y)$  de um ponto na curva. Pode ser representada em formato não comprimido (65 bytes, prefixo 0x04) ou comprimido (33 bytes, prefixo 0x02 se  $y$  é par, 0x03 se  $y$  é ímpar). A representação comprimida é possível porque a curva é simétrica em relação ao eixo  $x$ : dados  $x$  e a paridade de  $y$ , o valor completo de  $y$  pode ser reconstruído Bashir (2023). A partir da versão 0.6 do Bitcoin Core, chaves comprimidas tornaram-se o padrão, reduzindo em quase 50% o espaço ocupado pelas chaves na blockchain.

As chaves públicas são visíveis a todos os participantes da rede. Quando uma transação assinada com a chave privada é transmitida, os nós da rede utilizam a chave pública correspondente para verificar a assinatura e, portanto, comprovar a propriedade dos bitcoins.

### 5.2.3 Endereço Bitcoin

O endereço Bitcoin é obtido a partir da chave pública mediante uma cadeia de operações criptográficas Bashir (2023):

1. Geração aleatória de uma chave privada ECDSA.
2. Derivação da chave pública a partir da chave privada.
3. Aplicação de SHA-256 sobre a chave pública.
4. Aplicação de RIPEMD-160 sobre o resultado anterior.
5. Prefixação de um byte de versão ao hash RIPEMD-160.
6. Dupla aplicação de SHA-256 sobre o resultado.
7. Os primeiros 4 bytes do hash final constituem o *checksum*.
8. O *checksum* é acrescentado ao hash RIPEMD-160 do passo 4.
9. O resultado é codificado em Base58Check, produzindo o endereço final.

Os endereços resultantes possuem entre 26 e 35 caracteres e iniciam com o dígito 1 (P2PKH) ou 3 (P2SH). A codificação Base58 exclui caracteres visualmente ambíguos como 0/O e 1/I. Endereços de vaidade (*vanity addresses*) podem ser gerados por força bruta para conter sequências legíveis. Endereços multisignatura (M-de-N) exigem pelo menos  $M$  assinaturas de um conjunto de  $N$  chaves para liberar os fundos.

## 5.2.4 Custódia e Carteiras

Do ponto de vista da custódia, a segurança do Bitcoin é, em última análise, a segurança da chave privada. O software de carteira gera, armazena e emprega as chaves privadas para assinar transações. Existem diversas categorias de carteiras, detalhadas na Seção 5.8, mas o princípio fundamental é imutável: sem a chave privada, não há acesso aos bitcoins; com ela, o acesso é irrestrito.

## 5.3 Estrutura do Bloco Bitcoin

A blockchain do Bitcoin é um ledger público, distribuído e ordenado cronologicamente, que registra de forma imutável todas as transações da rede Bashir (2023). Cada bloco é composto por um cabeçalho e um corpo de transações.

### 5.3.1 Cabeçalho do Bloco

O cabeçalho possui 80 bytes fixos e contém seis campos Bashir (2023):

- **Version** (4 bytes): número de versão que indica as regras de validação a serem aplicadas.
- **Previous Block Header Hash** (32 bytes): hash SHA-256 duplo do cabeçalho do bloco imediatamente anterior. É este campo que encadeia os blocos e confere imutabilidade retroativa.
- **Merkle Root Hash** (32 bytes): hash SHA-256 duplo da raiz da Árvore de Merkle construída sobre todas as transações do bloco. Qualquer alteração em uma transação modifica a raiz e invalida o cabeçalho.
- **Timestamp** (4 bytes): tempo aproximado de criação do bloco em formato Unix epoch.
- **Bits** (4 bytes): representação compacta do alvo de dificuldade atual da rede.
- **Nonce** (4 bytes): número variável que os mineradores incrementam iterativamente até que o hash do cabeçalho seja inferior ao alvo de dificuldade.

### 5.3.2 Corpo do Bloco e a Transação Coinbase

O corpo do bloco contém o contador de transações (1–9 bytes) seguido das transações em si. A primeira transação de qualquer bloco é obrigatoriamente a *coinbase transaction*, criada pelo minerador, que emite novas moedas e recolhe as taxas de transação do bloco. A coinbase não referencia nenhuma saída anterior e pode armazenar até 100

bytes de dados arbitrários — espaço utilizado por Nakamoto para inserir a manchete do *The Times* no bloco gênese. Por regra de consenso, os bitcoins cunhados por uma transação coinbase não podem ser gastos antes que 100 blocos tenham sido acrescentados sobre ela.

### 5.3.3 Encadeamento e Imutabilidade

O encadeamento de blocos por referência ao hash do cabeçalho anterior é o mecanismo fundamental de imutabilidade da blockchain. Modificar qualquer transação em um bloco altera o Merkle root daquele bloco, o que altera seu hash, o que invalida o campo *Previous Block Header Hash* do bloco seguinte, e assim por diante em efeito cascata. Para adulterar uma transação passada, seria necessário refazer a Prova de Trabalho de todos os blocos subsequentes mais rapidamente do que a rede honesta acrescenta novos blocos — um feito computacionalmente inviável enquanto nenhum ator controle mais de 50% do poder computacional da rede Bashir (2023).

## 5.4 Transações e o Modelo UTXO

### 5.4.1 O Modelo UTXO

O Bitcoin não utiliza saldos de conta no sentido tradicional. Em vez disso, adota o modelo de *Unspent Transaction Outputs* (UTXO) Bashir (2023). Uma UTXO é a saída de uma transação que ainda não foi consumida por nenhuma entrada subsequente. O “saldo” de um endereço é a soma das UTXOs associadas às suas chaves. Quando um usuário quer enviar bitcoins, seleciona UTXOs como entradas, especifica saídas (destinatário e valor), e a diferença entre a soma das entradas e a soma das saídas constitui a taxa de transação paga ao minerador:

$$\text{taxa} = \sum \text{entradas} - \sum \text{saídas}$$

A menor unidade do Bitcoin é o Satoshi, equivalente a  $10^{-8}$  BTC.

### 5.4.2 Estrutura de uma Transação

Uma transação Bitcoin contém, em alto nível Bashir (2023):

- **Version** (4 bytes): versão da transação (1 ou 2).
- **Flag** (2 bytes ou ausente): indica a presença de dados de testemunha (*witness*), introduzidos pelo SegWit.

- **Input counter** (1–9 bytes) e lista de entradas: cada entrada referencia o hash da transação anterior e o índice da UTXO a ser consumida, e inclui o *ScriptSig* (script de desbloqueio).
- **Output counter** (1–9 bytes) e lista de saídas: cada saída especifica um valor em Satoshis e um *ScriptPubKey* (script de bloqueio).
- **Witness**: dados de testemunha por entrada, presentes apenas em transações Seg-Wit.
- **Lock time** (4 bytes): define o momento mais cedo em que a transação se torna válida (timestamp Unix ou altura de bloco).

### 5.4.3 Scripts e P2PKH

O Bitcoin emprega uma linguagem de script baseada em pilha, chamada *Script*, inspirada na linguagem Forth. Não é Turing-completa e não possui laços, o que evita programas de execução infinita na rede Bashir (2023). O tipo de transação mais comum é o P2PKH (*Pay-to-Public-Key Hash*):

- **ScriptPubKey (bloqueio)**: `OP_DUP OP_HASH160 <pubKeyHash> OP_EQUALVERIFY OP_CHECKSIG`
- **ScriptSig (desbloqueio)**: `<sig> <pubKey>`

A validação consiste em executar a concatenação `ScriptSig || ScriptPubKey` na pilha: o `OP_DUP` duplica a chave pública, `OP_HASH160` produz seu hash, `OP_EQUALVERIFY` compara esse hash com o hash do endereço de destino, e `OP_CHECKSIG` verifica a assinatura. Se todas as operações resultam em `TRUE`, a transação é válida.

Outros tipos padrão incluem P2SH (*Pay-to-Script Hash*), multisig (M-de-N), e `OP_RETURN` para armazenamento de dados arbitrários na cadeia (limitado a 40 bytes por saída).

### 5.4.4 Ciclo de Vida de uma Transação e a Mempool

Ao ser criada e assinada pelo remetente, a transação é transmitida à rede Bitcoin por um algoritmo de inundação (*flooding*). Cada nó a recebe, valida e a armazena temporariamente em sua *mempool* (*memory pool*) — um buffer em RAM que mantém as transações confirmadas mas ainda não incluídas em um bloco Bashir (2023). Os mineradores selecionam transações da mempool para compor o bloco candidato, priorizando aquelas com maior taxa por byte. Uma transação é considerada definitivamente confirmada após seis confirmações (seis blocos minerados sobre o bloco que a contém), momento em que a probabilidade de um ataque de duplo gasto torna-se estatisticamente desprezível.



## 5.5 Mineração e Prova de Trabalho

### 5.5.1 O Problema do Nonce e o Alvo de Dificuldade

A mineração é o processo pelo qual novos blocos são acrescentados à blockchain Bashir (2023). O núcleo do processo é a Prova de Trabalho (PoW), formalizada como:

$$H(N \parallel P_{\text{hash}} \parallel T_{x_1} \parallel \cdots \parallel T_{x_n}) < \text{Target}$$

onde  $N$  é o nonce,  $P_{\text{hash}}$  é o hash do bloco anterior,  $T_{x_i}$  são as transações do bloco e  $\text{Target}$  é o alvo de dificuldade. O minerador deve encontrar um valor de  $N$  tal que o hash duplo SHA-256 do cabeçalho do bloco seja numericamente inferior a  $\text{Target}$ . Não existe atalho: a única estratégia é força bruta, testando nonces sequencialmente. Como o nonce é um campo de apenas 32 bits ( $\approx 4,3 \times 10^9$  valores), e o espaço se esgota rapidamente nas taxas de hash modernas, implementou-se a solução do *extra nonce*: a transação coinbase é modificada para fornecer um campo adicional de nonce, expandindo dramaticamente o espaço de busca.

### 5.5.2 Ajuste de Dificuldade

A rede Bitcoin reajusta automaticamente a dificuldade a cada 2.016 blocos (aproximadamente duas semanas), segundo a fórmula Bashir (2023):

$$D_{\text{novo}} = D_{\text{anterior}} \times \frac{2016 \times 10 \text{ min}}{\text{tempo real para minerar os últimos 2016 blocos}}$$

O objetivo é manter o intervalo médio de geração de blocos em 10 minutos, independentemente da capacidade computacional agregada da rede. Se blocos estão sendo encontrados mais rapidamente, a dificuldade aumenta; se mais lentamente, diminui.

### 5.5.3 Recompensa e Halving

Os mineradores são recompensados com bitcoins recém-emitidos mais as taxas de transação do bloco. A emissão de novos bitcoins é reduzida pela metade a cada 210.000 blocos (aproximadamente 4 anos), em um evento chamado *halving*. Em 2009, a recompensa era 50 BTC; em maio de 2020, passou a 6,25 BTC. O halving seguinte, esperado em 2024, reduzirá a recompensa a 3,125 BTC. O fornecimento total de Bitcoin é limitado a 21 milhões de unidades, a serem completamente emitidas por volta de 2140. A partir desse ponto, os mineradores dependerão exclusivamente das taxas de transação como incentivo Bashir (2023).

### 5.5.4 Evolução do Hardware de Mineração

Ao longo da história do Bitcoin, a corrida por eficiência computacional levou a sucessivas gerações de hardware Bashir (2023):

1. **CPU:** Utilizada nos primórdios, até aproximadamente 2010. Qualquer computador pessoal podia minerar lucrativamente.
2. **GPU:** As unidades de processamento gráfico, programáveis em OpenCL, oferecem paralelismo massivo e superaram amplamente as CPUs em mineração.
3. **FPGA:** Circuitos programáveis em linguagens de descrição de hardware (Verilog, VHDL) ofereceram melhor eficiência energética, mas exigiram conhecimento especializado.
4. **ASIC:** Circuitos integrados de aplicação específica, projetados exclusivamente para executar SHA-256 duplo. Representam o estado da arte atual. Grandes centros de mineração operam milhares de unidades ASIC em paralelo e comercializam contratos de mineração.

### 5.5.5 Pools de Mineração

A probabilidade de um minerador individual encontrar um bloco é proporcional à sua fração do hash rate total da rede. Com a escalada da dificuldade, a mineração solo tornou-se economicamente inviável para a maioria. As *mining pools* agregam o poder computacional de múltiplos mineradores, que trabalham conjuntamente e dividem a recompensa proporcionalmente à contribuição de cada um (modelo proporcional) ou por taxa fixa por unidade de trabalho submetida (modelo *pay-per-share*) Bashir (2023).

A concentração de pools representa um risco sistêmico. Se uma pool conseguir controlar mais de 51% do hash rate, ela pode executar ataques de duplo gasto e reorganizar a cadeia. Esse cenário materializou-se em 2014, quando o GHash.io momentaneamente ultrapassou 51% do hash rate da rede Bitcoin.

## 5.6 A Cadeia: Encadeamento, Imutabilidade e Reorganização

### 5.6.1 Encadeamento por Hash

A propriedade central da blockchain é o encadeamento criptográfico: cada bloco referencia o hash do cabeçalho do bloco anterior. O bloco gênese (bloco 0) é o único sem predecessor e está hardcoded no software Bitcoin Core Bashir (2023). A altura de um bloco é o número de blocos que o precedem na cadeia principal.

## 5.6.2 Forks e Reorganização

Devido à natureza distribuída da rede, dois mineradores podem encontrar blocos válidos quase simultaneamente, criando um *fork* temporário com duas versões competindo pela continuidade da cadeia Bashir (2023). A regra de escolha de cadeia do Bitcoin seleciona a cadeia com maior trabalho acumulado (não simplesmente a mais longa em número de blocos). A cadeia mais curta é abandonada; seus blocos tornam-se *blocos órfãos* e seus mineradores não recebem recompensa.

Existem também forks deliberados decorrentes de mudanças de protocolo:

- **Soft fork:** Mudança retrocompatível. Nós que não atualizam continuam operando normalmente, pois os novos blocos também são válidos segundo as regras antigas. Apenas os mineradores precisam atualizar para aplicar as novas regras.
- **Hard fork:** Mudança incompatível com versões anteriores. Nós não atualizados rejeitam os novos blocos, resultando em duas blockchains permanentemente divergentes.

Blocos obsoletos (*stale blocks*) diferem dos órfãos: são blocos que foram minerados em uma cadeia que a rede já ultrapassou, mas cujo histórico parental é conhecido.

## 5.7 A Rede Peer-to-Peer

### 5.7.1 Descoberta e Propagação

O Bitcoin opera sobre uma rede P2P onde nós transmitem transações e blocos entre si Bashir (2023). Ao iniciar, um novo nó consulta sementes DNS (*DNS seeds*) hardcoded no Bitcoin Core para descobrir pares. A comunicação ocorre via TCP, porta 8333 na rede principal e 18333 na testnet. O *handshake* inicial envolve a troca das mensagens de protocolo `version` e `verack`, seguida de `getaddr/addr` para descoberta de pares adicionais.

A partir da versão 0.10.0, o Bitcoin Core adotou o método *headers-first* para o download inicial da blockchain: o nó primeiro solicita e valida os cabeçalhos de bloco de todos os pares disponíveis e, em seguida, requisita os blocos completos em paralelo, acelerando significativamente a sincronização.

### 5.7.2 Tipos de Nós

Há dois tipos principais de nós Bashir (2023):

- **Nós completos (Full nodes):** Armazenam a blockchain completa, validam independentemente todas as transações e blocos, e propagam dados à rede. Constituem o bastião de segurança do sistema.
- **Nós SPV (Simplified Payment Verification):** Mantêm apenas os cabeçalhos de bloco. Para verificar uma transação, solicitam a prova de Merkle do bloco correspondente a um nó completo. São adequados para dispositivos com recursos limitados, como smartphones, mas dependem de nós completos para a validação plena.

Nós de pool de mineração utilizam o protocolo Stratum, baseado em TCP com JSON-RPC legível, para coordenar o trabalho entre o servidor da pool e os mineradores individuais.

### 5.7.3 Bloom Filters e Privacidade em SPV

Para que nós SPV possam requisitar apenas transações de seu interesse sem revelar seus endereços, o BIP37 introduziu os *bloom filters* Bashir (2023). Um bloom filter é uma estrutura de dados probabilística que permite testar a pertinência de um elemento a um conjunto com possibilidade de falsos positivos, mas sem falsos negativos. O SPV configura um filtro que engloba seus endereços e, ao solicitar transações, recebe um superconjunto que inclui as relevantes mais alguns falsos positivos, dificultando a inferência de quais endereços pertencem ao nó. Entretanto, filtros muito restritivos ainda podem revelar informações suficientes para correlacionar endereços a endereços IP.

## 5.8 Carteiras Bitcoin

### 5.8.1 Tipos de Carteiras

O software de carteira gerencia chaves criptográficas e abstrai as operações de assinatura de transações para o usuário Bashir (2023). As principais categorias são:

- **Carteiras não determinísticas:** Geram chaves privadas independentemente de forma aleatória (“*Just a Bunch of Keys*”). A gestão de múltiplas chaves é complexa e propensa a erros.
- **Carteiras determinísticas:** Derivam todas as chaves a partir de um único valor semente (*seed*), geralmente representado como uma sequência de palavras mnemônicas. Basta preservar a semente para recuperar todas as chaves.
- **Carteiras HD (Hierarchical Deterministic):** Definidas nos BIPs 32 e 44, organizam as chaves em uma estrutura hierárquica em árvore derivada de uma chave mestre (*master key*). A profundidade da hierarquia permite separar logicamente

diferentes fins (contas, endereços de recebimento vs. de troco, etc.) enquanto toda a estrutura é recuperável a partir de uma única semente.

- **Carteiras de papel:** A chave privada e o endereço são impressos em papel. Simples, mas exigem segurança física rigorosa.
- **Carteiras de hardware:** Dispositivos físicos resistentes a adulteração (por exemplo, Trezor e Ledger) que armazenam as chaves em um elemento seguro e assinam transações internamente, sem expor a chave privada ao computador do host. Representam o padrão-ouro de segurança para uso cotidiano.
- **Carteiras online (hot wallets):** Armazenadas e acessadas via internet. Convenientes, mas expõem as chaves a riscos de segurança da plataforma provedora.
- **Carteiras móveis:** Instaladas em smartphones. Populares por permitir o escaneamento de QR codes e pagamentos instantâneos, com bom equilíbrio entre usabilidade e segurança.
- **Brain wallets:** A chave mestre é derivada de uma frase memorizada pelo usuário. Extremamente vulnerável a ataques de dicionário e força bruta.

### 5.8.2 HD Wallets, BIP32/39/44 e Seed Phrases

O ecossistema de HD wallets é definido por três BIPs complementares Bashir (2023):

- **BIP32:** Especifica a derivação hierárquica de chaves a partir de uma chave mestre. Define tanto chaves de derivação normal (*unhardened*) quanto chaves endurecidas (*hardened*), que não podem ser comprometidas mesmo se uma chave filha pública for exposta.
- **BIP39:** Estabelece o padrão de mnemônicos: a semente é codificada como 12 ou 24 palavras em linguagem natural a partir de uma lista padronizada de 2048 palavras. Essa *seed phrase* (frase semente) permite a recuperação completa da carteira.
- **BIP44:** Define caminhos de derivação padronizados para múltiplas moedas e contas, com a estrutura `m / purpose' / coin_type' / account' / change / address_index`.

A distinção fundamental entre armazenamento *hot* (chaves conectadas à internet) e *cold* (chaves isoladas da internet) é crítica para a gestão de risco. Fundos significativos devem ser mantidos em cold storage, idealmente em carteiras de hardware, enquanto quantias menores para uso frequente podem residir em carteiras móveis ou de software.

## 5.9 Limitações e Críticas ao Bitcoin

Apesar de sua resiliência e impacto histórico, o Bitcoin apresenta limitações técnicas e estruturais significativas que motivaram o surgimento de plataformas blockchain alternativas Bashir (2023).

### 5.9.1 Throughput e Escalabilidade

O Bitcoin processa, em média, cerca de 7 transações por segundo (TPS) na camada base. Isso contrasta dramaticamente com sistemas de pagamento tradicionais: a rede Visa, por exemplo, suporta dezenas de milhares de TPS. O gargalo decorre de três parâmetros interligados: o tamanho máximo de bloco (limitado originalmente a 1 MB, expandido de forma efetiva pelo SegWit), o intervalo de 10 minutos entre blocos, e o número de transações que cabem em cada bloco.

Soluções de segunda camada, como a *Lightning Network*, propõem canais de pagamento fora da cadeia que liquidam periodicamente na blockchain, permitindo transações quase instantâneas e com taxas mínimas. Entretanto, adoção, liquidez e complexidade operacional ainda são desafios.

### 5.9.2 Consumo Energético

O PoW exige que os mineradores consumam grandes quantidades de energia elétrica em competição. O consumo anual da rede Bitcoin é comparável ao de países de porte médio, gerando críticas consideráveis quanto à sustentabilidade ambiental. Embora parte da mineração utilize fontes renováveis, o debate sobre o custo ecológico do PoW permanece relevante e influenciou a adoção de mecanismos alternativos de consenso, como a Prova de Participação (*Proof of Stake*), em outras blockchains.

### 5.9.3 Programabilidade Limitada

A linguagem Script do Bitcoin é intencionalmente restrita: não é Turing-completa e não suporta laços ou estado persistente arbitrário. Isso limita os tipos de contratos que podem ser expressos. Contratos de custódia, canais de pagamento e alguns esquemas multisig são viáveis, mas contratos mais sofisticados — como os *smart contracts* da plataforma Ethereum — estão fora do alcance nativo do Bitcoin Bashir (2023). O projeto Miniscript busca tornar a escrita de scripts mais estruturada e verificável, mas não elimina a restrição fundamental de completude computacional.

### 5.9.4 Privacidade

A blockchain do Bitcoin é um ledger público: todas as transações são visíveis a qualquer observador. Embora os endereços sejam pseudônimos (não nomes reais), técnicas de análise de cadeia (*chain analysis*) permitem correlacionar endereços, rastrear fluxos de fundos e, em muitos casos, deanonimizar usuários. Soluções como CoinJoin, PayJoin e a proposta de endereços *stealth* melhoram a privacidade, mas não são parte do protocolo base.

### 5.9.5 Vulnerabilidades e Bugs Históricos

Dois incidentes notáveis ilustram vulnerabilidades do protocolo Bashir (2023): o bug de *value overflow* em agosto de 2010, que criou aproximadamente 184 bilhões de bitcoins devido a um estouro de inteiro, corrigido rapidamente via soft fork; e a maleabilidade de transações (*transaction malleability*), que permitia alterar o identificador de uma transação antes de sua confirmação, criando ambiguidade sobre sua execução — vetor explorado em alguns ataques a exchanges e corrigido pelo BIP62 e, posteriormente, pelo SegWit.

## Capítulo 6

# Contratos Inteligentes, Aplicações Descentralizadas e Oráculos

### 6.1 Da Blockchain como Registro à Blockchain Programável

A evolução da tecnologia blockchain pode ser compreendida em uma progressão conceitual: primeiro, blockchain como sistema distribuído; depois, blockchain como rede descentralizada; em seguida, blockchain como estrutura criptográfica de validação; por fim, blockchain como ambiente de execução programável. Essa passagem é essencial para compreender por que a tecnologia ultrapassou o caso de uso original do Bitcoin. O Bitcoin demonstrou que era possível manter um histórico monetário compartilhado sem uma autoridade central; os contratos inteligentes ampliaram essa ideia ao permitir que regras de negócio fossem executadas sobre um estado replicado, transparente e resistente a adulterações Bashir (2023).

O termo *smart contract* antecede a blockchain. Nick Szabo já o utilizava para descrever protocolos eletrônicos de transação capazes de executar termos contratuais, reduzir dependência de intermediários e diminuir custos de fraude, arbitragem e execução Szabo (1997). No contexto das blockchains modernas, um contrato inteligente pode ser definido como um programa armazenado e executado em uma infraestrutura distribuída, cujas regras são aplicadas automaticamente quando determinadas condições são satisfeitas. A diferença em relação a um programa tradicional não está apenas no código, mas no ambiente: sua execução é validada por uma rede, sua lógica é auditável e seus efeitos alteram um estado compartilhado.



## 6.2 Propriedades dos Contratos Inteligentes

Um contrato inteligente útil em blockchain deve apresentar algumas propriedades fundamentais Bashir (2023). A primeira é o **determinismo**: todos os nós que executam o contrato, partindo do mesmo estado inicial e recebendo a mesma entrada, devem obter o mesmo resultado. Sem determinismo, o consenso sobre o novo estado da blockchain seria impossível. A segunda é a **execução automática**: uma vez implantado e acionado, o contrato não depende de intervenção humana para aplicar sua lógica. A terceira é a **resistência a alterações**: depois de publicado em uma blockchain pública, o código tende a ser imutável, salvo quando o próprio projeto prevê mecanismos explícitos de atualização.

Essas propriedades trazem benefícios, mas também riscos. A execução automática reduz fricção e dependência de intermediários, porém amplifica erros de programação. Um contrato mal escrito pode executar exatamente o que foi codificado, e não o que seus criadores pretendiam. Por isso, a segurança de contratos inteligentes exige auditoria, testes, análise formal e cuidado com padrões conhecidos de vulnerabilidade, como reentrância, estouro aritmético, controle de acesso incorreto e dependência inadequada de dados externos.

## 6.3 Smart Contracts no Bitcoin e no Ethereum

O Bitcoin possui uma linguagem de script propositalmente restrita. Ela permite expressar condições de gasto, como assinaturas múltiplas, travas temporais e regras simples de autorização. Essa limitação é uma escolha de projeto: reduzir a superfície de ataque e preservar a previsibilidade da rede. Entretanto, a ausência de completude de Turing, laços e estado persistente arbitrário limita a criação de aplicações complexas diretamente na camada base do Bitcoin Bashir (2023).

O Ethereum foi concebido para ocupar justamente esse espaço. Em vez de se restringir a transferências de valor e scripts simples, ele introduz uma máquina virtual capaz de executar contratos com estado próprio. Com isso, a blockchain deixa de ser apenas um livro-razão de transações e passa a operar como uma máquina de estados programável. Essa diferença é central para a análise: Bitcoin mostra como chaves, transações, blocos e Prova de Trabalho podem viabilizar dinheiro digital descentralizado; Ethereum mostra como a mesma família de ideias pode sustentar aplicações descentralizadas, tokens, DAOs e regras de negócio on-chain.

## 6.4 Aplicações Descentralizadas

Aplicações descentralizadas, ou DApps, combinam uma interface de usuário com contratos inteligentes e uma blockchain como infraestrutura de estado. Em uma aplicação tradicional, o backend costuma ser controlado por uma empresa ou servidor central. Em uma DApp, parte relevante da lógica de negócio é executada por contratos inteligentes, e o estado crítico é mantido por uma rede distribuída Bashir (2023).

Uma DApp não é automaticamente descentralizada apenas por usar uma blockchain. Seu grau real de descentralização depende de vários componentes: onde está hospedada a interface, como os dados são armazenados, quem controla as chaves administrativas dos contratos, como as atualizações são aprovadas, qual oráculo fornece dados externos e como a governança do protocolo é exercida. Assim, uma aplicação pode ter contratos em uma blockchain pública, mas continuar dependente de infraestrutura centralizada em sua interface, em seus servidores auxiliares ou em sua governança.

Apesar desses limites, DApps representam uma mudança arquitetural relevante. Elas permitem criar mercados, sistemas de votação, protocolos financeiros, jogos, plataformas de identidade e mecanismos de governança em que regras fundamentais são transparentes e verificáveis. Essa característica dialoga diretamente com a ideia de Web 3: uma web na qual identidade, ativos digitais e regras de interação dependem menos de plataformas centralizadas e mais de protocolos abertos.

## 6.5 DAOs e o Problema da Governança

Organizações autônomas descentralizadas, ou DAOs, são estruturas organizacionais que utilizam contratos inteligentes para coordenar decisões, tesouraria e regras de participação. Em uma DAO, direitos de voto, propostas, quóruns e execução de decisões podem ser codificados e auditados publicamente. O objetivo é reduzir a dependência de uma administração central e permitir que a comunidade governe o protocolo ou projeto de forma transparente Bashir (2023).

O caso da The DAO, em 2016, tornou-se um marco histórico. A iniciativa arrecadou grande volume de recursos, mas uma vulnerabilidade de reentrância permitiu que um atacante drenasse fundos antes que o contrato atualizasse corretamente seu estado interno. A resposta da comunidade levou a um hard fork no Ethereum para recuperar os valores, enquanto parte dos participantes manteve a cadeia original, conhecida como Ethereum Classic. O episódio é didático porque evidencia uma tensão profunda: se “código é lei”, o que fazer quando o código contém erro? A resposta prática do ecossistema mostrou que, mesmo em sistemas descentralizados, governança social, valores comunitários e decisões políticas continuam existindo.

## 6.6 Oráculos e a Ponte com o Mundo Real

Contratos inteligentes são eficientes para lidar com dados já existentes na blockchain, mas não acessam diretamente eventos externos. Eles não sabem, por si mesmos, o preço de um ativo, o resultado de uma eleição, a temperatura medida por um sensor, o atraso de um voo ou a cotação de uma moeda. Oráculos surgem como mecanismos para levar dados externos para dentro da blockchain ou para comunicar eventos da blockchain a sistemas externos Bashir (2023).

O problema dos oráculos é que eles podem reintroduzir confiança em terceiros. Um contrato descentralizado que depende de um único provedor de dados passa a herdar o risco desse provedor. Se o oráculo falhar, for manipulado ou sofrer ataque, o contrato pode executar decisões economicamente incorretas. Por isso, soluções mais robustas utilizam agregação de múltiplas fontes, incentivos criptoeconômicos, reputação, penalidades e mecanismos de verificação independente.

Há diferentes tipos de oráculos. Oráculos de entrada levam dados externos para contratos; oráculos de saída comunicam eventos on-chain a sistemas externos; oráculos de hardware integram sensores e dispositivos físicos; oráculos computacionais executam cálculos fora da cadeia; e oráculos descentralizados combinam múltiplas fontes para reduzir dependência de um ponto único de falha. Essa discussão é crucial porque mostra que a blockchain não elimina todos os problemas de confiança: muitas vezes ela desloca a confiança para fronteiras específicas, que precisam ser projetadas com o mesmo rigor do protocolo base.

# Capítulo 7

## Arquitetura do Ethereum

### 7.1 Ethereum como Máquina de Estados Programável

O Ethereum é uma blockchain pública projetada para executar programas. Sua contribuição central consiste em generalizar a ideia de blockchain: em vez de manter apenas um histórico de transferências monetárias, a rede mantém um estado global que pode ser alterado por transações e contratos inteligentes. Cada nó participante valida não apenas a ordem das transações, mas também se a transição de estado produzida pela execução dessas transações é correta Bashir (2023).

Essa arquitetura pode ser compreendida como uma máquina de estados replicada. O estado atual contém contas, saldos, códigos de contratos e armazenamento persistente. Uma transação válida atua como entrada. A Ethereum Virtual Machine executa a lógica correspondente e produz um novo estado. O consenso garante que a rede concorde sobre qual sequência de transações e quais transições de estado formam a história canônica. Portanto, Ethereum não é apenas uma criptomoeda, mas uma máquina virtual distribuída.

### 7.2 Contas, Chaves e Endereços

Ethereum utiliza criptografia de chave pública para representar controle sobre contas. Uma chave privada gera uma chave pública, da qual se deriva um endereço. O controle da chave privada permite assinar transações e autorizar ações na rede. Esse princípio é semelhante ao Bitcoin: quem controla a chave controla os ativos e as ações possíveis associadas ao endereço.

A diferença relevante está no modelo de contas. Ethereum possui dois tipos principais Bashir (2023): **contas externamente controladas** (*Externally Owned Accounts*, EOAs), controladas por chaves privadas de usuários; e **contas de contrato**, associadas a código e armazenamento próprio. Uma EOA pode enviar transações, transferir Ether ou chamar funções de contratos. Uma conta de contrato não inicia transações por conta

própria; ela executa código quando é chamada por uma transação ou por outro contrato.

Esse modelo contrasta com o UTXO do Bitcoin. No Bitcoin, valores são representados como saídas não gastas que são consumidas e criadas por transações. No Ethereum, saldos e estados são mantidos em contas. Essa escolha facilita a programação de contratos, pois cada contrato possui armazenamento persistente que pode ser lido e modificado conforme regras definidas em seu código.

## 7.3 Transações, Mensagens e Gas

Transações em Ethereum podem transferir Ether, criar contratos ou chamar funções de contratos existentes. Elas são assinadas por contas externas e propagadas pela rede. Durante a execução, contratos também podem gerar chamadas internas, usualmente chamadas de mensagens. A distinção é importante: transações entram na rede vindas de usuários; mensagens são efeitos internos da execução de contratos Bashir (2023).

O gas é o mecanismo que torna a computação mensurável e economicamente limitada. Cada operação da EVM possui um custo. Ao enviar uma transação, o usuário define um limite de gas e uma disposição de pagamento. Se a execução consumir todo o gas disponível, ela falha e as alterações de estado são revertidas, embora o gas gasto seja cobrado. Esse mecanismo evita loops infinitos e ataques de negação de serviço, ao mesmo tempo em que cria um mercado para o uso dos recursos computacionais da rede.

Do ponto de vista conceitual, o gas é uma solução elegante para um problema difícil: como permitir execução programável em uma rede aberta sem permitir que um contrato consuma recursos indefinidamente? A resposta é transformar cada passo computacional em custo econômico explícito.

## 7.4 Ethereum Virtual Machine

A Ethereum Virtual Machine (EVM) é o ambiente responsável por executar contratos inteligentes. Ela é determinística, isolada e baseada em uma arquitetura de pilha. Contratos escritos em linguagens de alto nível, como Solidity ou Vyper, são compilados para bytecode, que é interpretado pela EVM. Durante a execução, a EVM manipula áreas como stack, memória temporária, calldata e storage persistente Bashir (2023).

O determinismo da EVM é indispensável. Todos os nós devem obter o mesmo resultado ao executar o mesmo bytecode sob o mesmo estado inicial. Por isso, contratos não podem depender diretamente de fontes externas arbitrarias, relógios locais ou chamadas de rede. Quando precisam de informação externa, recorrem a oráculos. Essa limitação não é um defeito acidental; é uma condição para que a rede consiga chegar a consenso sobre resultados computacionais.

## 7.5 Estruturas de Dados e Estado

Além de blocos e transações, Ethereum precisa representar estado, recibos e provas de execução. Para isso, utiliza estruturas criptográficas derivadas de árvores de Merkle, como tries, que permitem resumir grandes conjuntos de dados em raízes criptográficas. Essas raízes aparecem nos cabeçalhos de bloco e permitem verificar, de forma compacta, transações, recibos e estado Bashir (2023).

A complexidade dessas estruturas é maior que no Bitcoin porque Ethereum não verifica apenas se uma transferência de valor é válida. Ele precisa verificar o resultado da execução de programas. Assim, a rede mantém uma relação permanente entre transações, recibos, logs de eventos, código de contratos e armazenamento persistente. Essa arquitetura torna possível construir aplicações mais complexas, mas também aumenta os desafios de escalabilidade, armazenamento e sincronização.

## 7.6 Consenso e a Transição para Proof of Stake

O livro de Bashir descreve a arquitetura original do Ethereum com base em Prova de Trabalho, incluindo mineração, Ethash, blocos e omers. Esse conteúdo é historicamente importante, mas a rede Ethereum mudou profundamente em setembro de 2022, quando ocorreu o evento conhecido como *The Merge*. A partir dele, a Ethereum Mainnet passou a utilizar Prova de Participação, substituindo mineradores por validadores que bloqueiam Ether em *staking* e participam da proposição e atestação de blocos Ethereum Foundation (2022).

Essa mudança preserva o ponto conceitual mais importante: o consenso continua servindo para ordenar transações e garantir acordo sobre o estado canônico da rede. O que muda é a fonte de segurança. Em PoW, a segurança vem do custo computacional e energético. Em PoS, ela vem do capital em risco: validadores podem ser recompensados por comportamento correto e penalizados, inclusive com perda de parte do stake, por comportamento malicioso ou negligente.

Na perspectiva do trilema blockchain, PoS não elimina os compromissos entre descentralização, segurança e escalabilidade, mas altera o equilíbrio entre eles. Reduz o consumo energético e permite novas formas de finalização econômica, porém introduz desafios próprios, como concentração de stake, governança de clientes, risco de censura por validadores e dependência de infraestrutura de staking.

## 7.7 Tokens, Padrões e Ecossistema

Um dos efeitos mais importantes do Ethereum foi padronizar a criação de ativos digitais programáveis. O padrão ERC-20 viabilizou tokens fungíveis, úteis para moedas de

protocolo, stablecoins, governança e representações digitais de valor. O ERC-721 tornou popular a ideia de tokens não fungíveis (NFTs), nos quais cada unidade possui identidade própria. O ERC-1155 combinou flexibilidade para representar múltiplos tipos de tokens em um mesmo contrato Bashir (2023).

Esses padrões são relevantes porque demonstram a força da composabilidade. Uma vez que carteiras, exchanges, protocolos e interfaces entendem um padrão, novos contratos podem interoperar com o ecossistema existente. Essa característica explica parte do crescimento de DeFi, NFTs e DAOs: aplicações diferentes podem se conectar como blocos de construção, desde que respeitem interfaces comuns.

# Capítulo 8

## Conclusão

Esta monografia apresentou blockchain como uma tecnologia de sistemas distribuídos, e não apenas como sinônimo de criptomoeda. A análise começou pela ideia de estado compartilhado entre nós independentes, passou pelos conceitos de descentralização, rede ponto-a-ponto, criptografia e consenso, e então aplicou esses fundamentos à arquitetura do Bitcoin e do Ethereum.

O estudo mostrou que o Bitcoin combinou contribuições anteriores de criptografia, funções de hash, assinaturas digitais, redes P2P e Prova de Trabalho para resolver o problema do gasto duplo sem depender de um servidor central. Nessa arquitetura, chaves privadas provam posse, transações movimentam valores, blocos organizam o histórico, mineradores competem para propor novos blocos e nós completos verificam as regras de forma independente.

Também foi possível observar que o Ethereum amplia essa lógica ao transformar a blockchain em uma máquina de estados programável. Com contratos inteligentes, EVM, gas e contas de contrato, a rede passa a executar regras sobre um estado compartilhado, permitindo aplicações descentralizadas, DAOs e integração com dados externos por meio de oráculos. Essa evolução não elimina os desafios do modelo, mas mostra por que blockchain pode ser entendida como infraestrutura de coordenação e execução distribuída.

Conclui-se que a importância da blockchain está na combinação entre replicação, consenso e verificação criptográfica. Seu valor não está em substituir todo banco de dados tradicional, mas em permitir que participantes que não confiam plenamente uns nos outros mantenham um histórico comum, válido e resistente a alterações unilaterais. Essa é a ideia central desenvolvida ao longo desta monografia.



# Referências

- BASHIR, Imran. **Mastering Blockchain: inner workings of blockchain, from cryptography and decentralized identities, to DeFi, NFTs and Web3**. 4. ed. Birmingham: Packt Publishing, 2023.
- BREWER, Eric A. Towards robust distributed systems. In: **Proceedings of the nineteenth annual ACM Symposium on Principles of Distributed Computing**. Portland: ACM, 2000.
- DWORK, Cynthia; NAOR, Moni. Pricing via processing or combatting junk mail. In: **Advances in Cryptology – CRYPTO ’92**. Berlin: Springer, 1992.
- ETHEREUM FOUNDATION. **The Merge**. 2022. Disponível em: <https://ethereum.org/en/roadmap/merge/>. Acesso em: 16 jun. 2026.
- GILBERT, Seth; LYNCH, Nancy. Brewer’s conjecture and the feasibility of consistent, available, partition-tolerant web services. **ACM SIGACT News**, v. 33, n. 2, p. 51–59, 2002.
- LAMPORT, Leslie; SHOSTAK, Robert; PEASE, Marshall. The Byzantine generals problem. **ACM Transactions on Programming Languages and Systems**, v. 4, n. 3, p. 382–401, 1982.
- MERKLE, Ralph C. A digital signature based on a conventional encryption function. In: **Advances in Cryptology – CRYPTO ’87**. Berlin: Springer, 1988.
- NAKAMOTO, Satoshi. **Bitcoin: a peer-to-peer electronic cash system**. 2008. Disponível em: <https://bitcoin.org/bitcoin.pdf>. Acesso em: 16 jun. 2026.
- SZABO, Nick. Formalizing and securing relationships on public networks. **First Monday**, v. 2, n. 9, 1997.
- WOOD, Gavin. **Ethereum: a secure decentralised generalised transaction ledger**. Ethereum Yellow Paper, 2014.